Chapter 1

Software Testing

ISTQB / ISEB Foundation Exam Practice

Principles of Testing

1 Principles	2 Lifecycle	3 Static testing
4 Test design techniques	5 Management	6 Tools

Principles

1	2	3
4	5	6

Contents

Why testing is necessary

What is testing

Testing principles

Fundamental test process

Psychology of testing

Code of Ethics

Testing terminology

- **No generally accepted set of testing definitions used world wide**
- **New standard BS 7925-1**
  - Glossary of testing terms (emphasis on component testing)
  - most recent
  - developed by a working party of the BCS SIGIST
  - adopted by the ISEB / ISTQB

What is a “bug”?

- **Error: a human action that produces an incorrect result**
- **Fault: a manifestation of an error in software**
  - also known as a defect or bug
  - if executed, a fault may cause a failure
- **Failure: deviation of the software from its expected delivery or service**
  - (found defect)

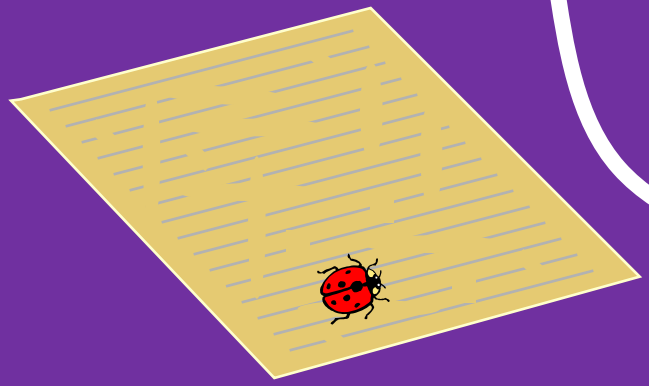
Failure is an event; fault is a state of the software, caused by an error

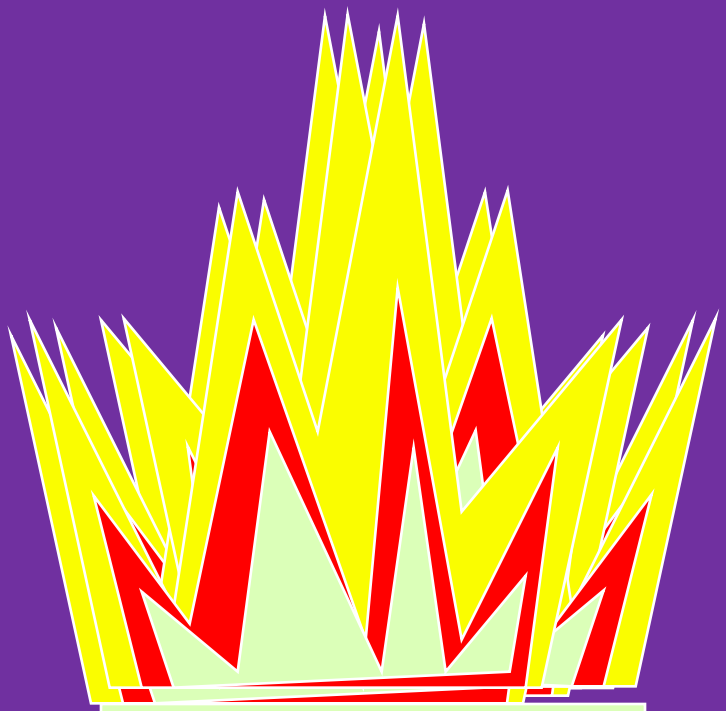
Error - Fault - Failure

A person makes an error ...



… that creates a fault in the software ...





… that can cause a failure in operation

Reliability versus faults

- **Reliability: the probability that software will not cause the failure of the system for a specified time under specified conditions**
  - Can a system be fault-free? (zero faults, right first time) ✗
  - Can a software system be reliable but still have faults? ✓
  - Is a “fault-free” software application always reliable? ✗

Why do faults occur in software?

- **software is written by human beings**
  - who know something, but not everything
  - who have skills, but aren't perfect
  - who do make mistakes (errors)
- **under increasing pressure to deliver to strict deadlines**
  - no time to check but assumptions may be wrong
  - systems may be incomplete
- **if you have ever written software ...**

What do software faults cost?

- **huge sums**
  - Ariane 5 ($7billion)
  - Mariner space probe to Venus ($250m)
  - American Airlines ($50m)
- **very little or nothing at all**
  - minor inconvenience
  - no visible or physical detrimental impact
- **software is not “linear”:**
  - small input may have very large effect

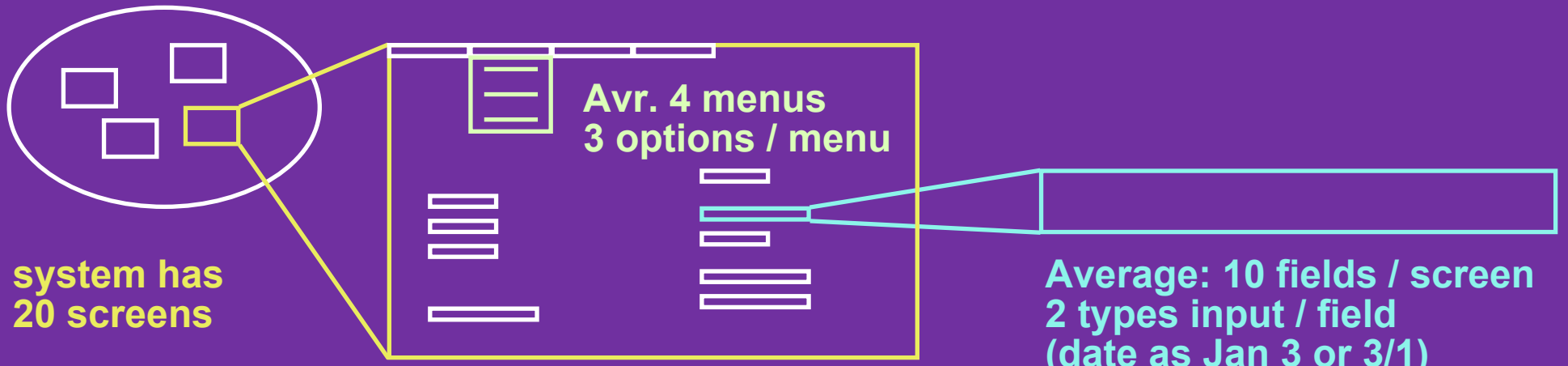
Safety-critical systems

- **software faults can cause death or injury**
  - radiation treatment kills patients (Therac-25)
  - train driver killed
  - aircraft crashes (Airbus & Korean Airlines)
  - bank system overdraft letters cause suicide

So why is testing necessary?

- because software is likely to have faults ✓
- to learn about the reliability of the software ✓
- to fill the time between delivery of the software and the release date ✗
- to prove that the software has no faults ✗
- because testing is included in the project plan ✗
- because failures can be very expensive ✓
- to avoid being sued by customers ✓
- to stay in business ✓

Why not just "test everything"?



Total for 'exhaustive' testing:

20 x 4 x 3 x 10 x 2 x 100 = 480,000 tests

If 1 second per test, 8000 mins, 133 hrs, 17.7 days

(not counting finger trouble, faults or retest)

10 secs = 34 wks, 1 min = 4 yrs, 10 min = 40 yrs

Exhaustive testing?

- **What is exhaustive testing?**
  - when all the testers are exhausted ✗
  - when all the planned tests have been executed ✗
  - exercising all combinations of inputs and preconditions ✓
- **How much time will exhaustive testing take?**
  - infinite time ✗
  - not much time ✗
  - impractical amount of time ✓

How much testing is enough?

- it's never enough ✗
- when you have done what you planned ✗
- when your customer/user is happy ✗
- when you have proved that the system works correctly ✗
- when you are confident that the system works correctly ✗ ✓
- it depends on the risks for your system ✓

How much testing?

- It depends on **RISK**
  - risk of missing important faults
  - risk of incurring failure costs
  - risk of releasing untested or under-tested software
  - risk of losing credibility and market share
  - risk of missing a market window
  - risk of over-testing, ineffective testing

So little time, so much to test ..

- **test time will always be limited**
- **use RISK to determine:**
  - what to test first
  - what to test most
  - how thoroughly to test each item
  - what not to test (this time)

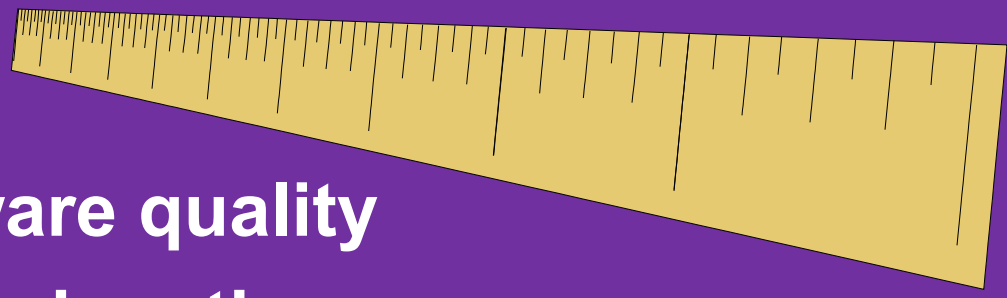
} **i.e. where to place emphasis**

- **use RISK to**
  - allocate the time available for testing by prioritising testing ...

Most important principle

Prioritise tests
so that,
whenever you stop testing,
you have done the best testing
in the time available.

Testing and quality



- **testing measures software quality**
- **testing can find faults; when they are removed, software quality (and possibly reliability) is improved**
- **what does testing test?**
  - system function, correctness of operation
  - non-functional qualities: reliability, usability, maintainability, reusability, testability, etc.

Other factors that influence testing

- **contractual requirements**
- **legal requirements**
- **industry-specific requirements**
  - e.g. pharmaceutical industry (FDA), compiler standard tests, safety-critical or safety-related such as railroad switching, air traffic control

It is difficult to determine
how much testing is enough
but it is not impossible

Principles

1	2	3
4	5	6

Contents

Why testing is necessary

What is testing

Testing principles

Fundamental test process

Psychology of testing

Code of Ethics

Seven Testing Principles

- Testing shows the presence of defects
- Exhaustive testing is impossible
- Early testing
- Defect clustering
- Pesticide paradox
- Testing is context dependent
- Absence-of-errors fallacy

Principles

1	2	3
4	5	6

Contents

Why testing is necessary

What is testing

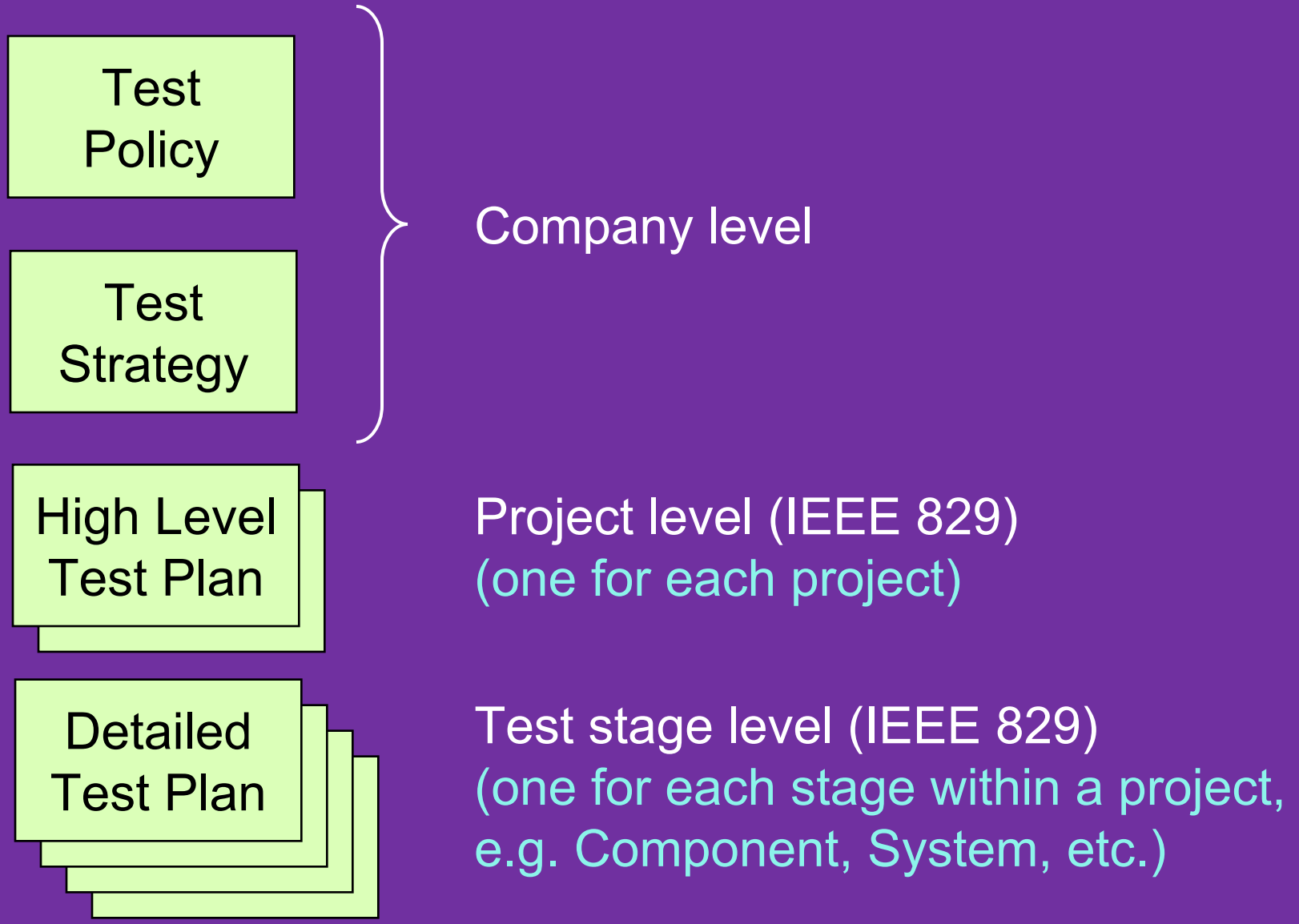
Testing principles

Fundamental test process

Psychology of testing

Code of Ethics

Test Planning - different levels



The test process

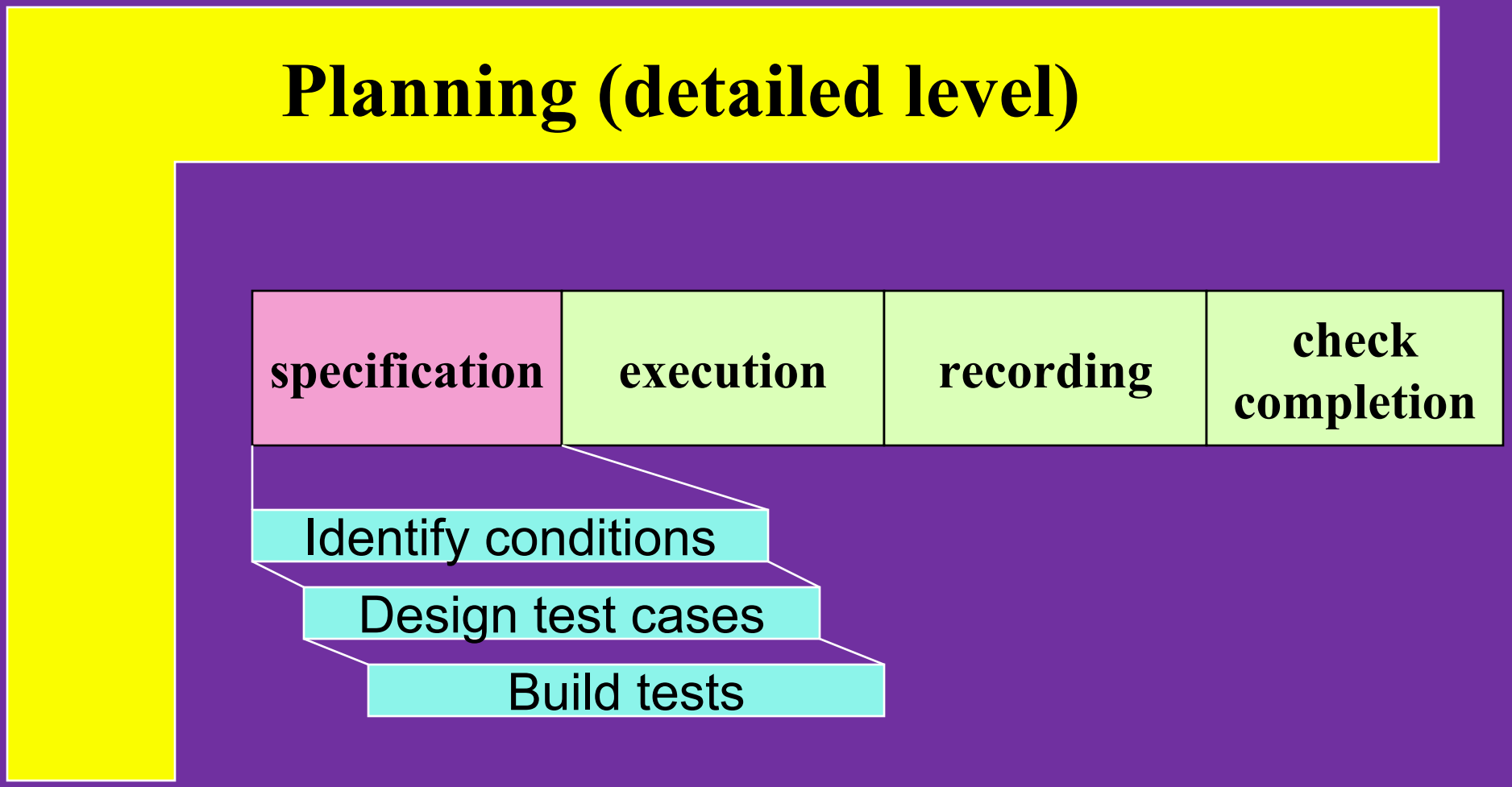
Planning (detailed level)

specification	execution	recording	check completion

Test planning

- how the test strategy and project test plan apply to the software under test
- document any exceptions to the test strategy
  - e.g. only one test case design technique needed for this functional area because it is less critical
- other software needed for the tests, such as stubs and drivers, and environment details
- set test completion criteria

Test specification



A good test case

- effective
- exemplary
- evolvable
- economic









Test specification

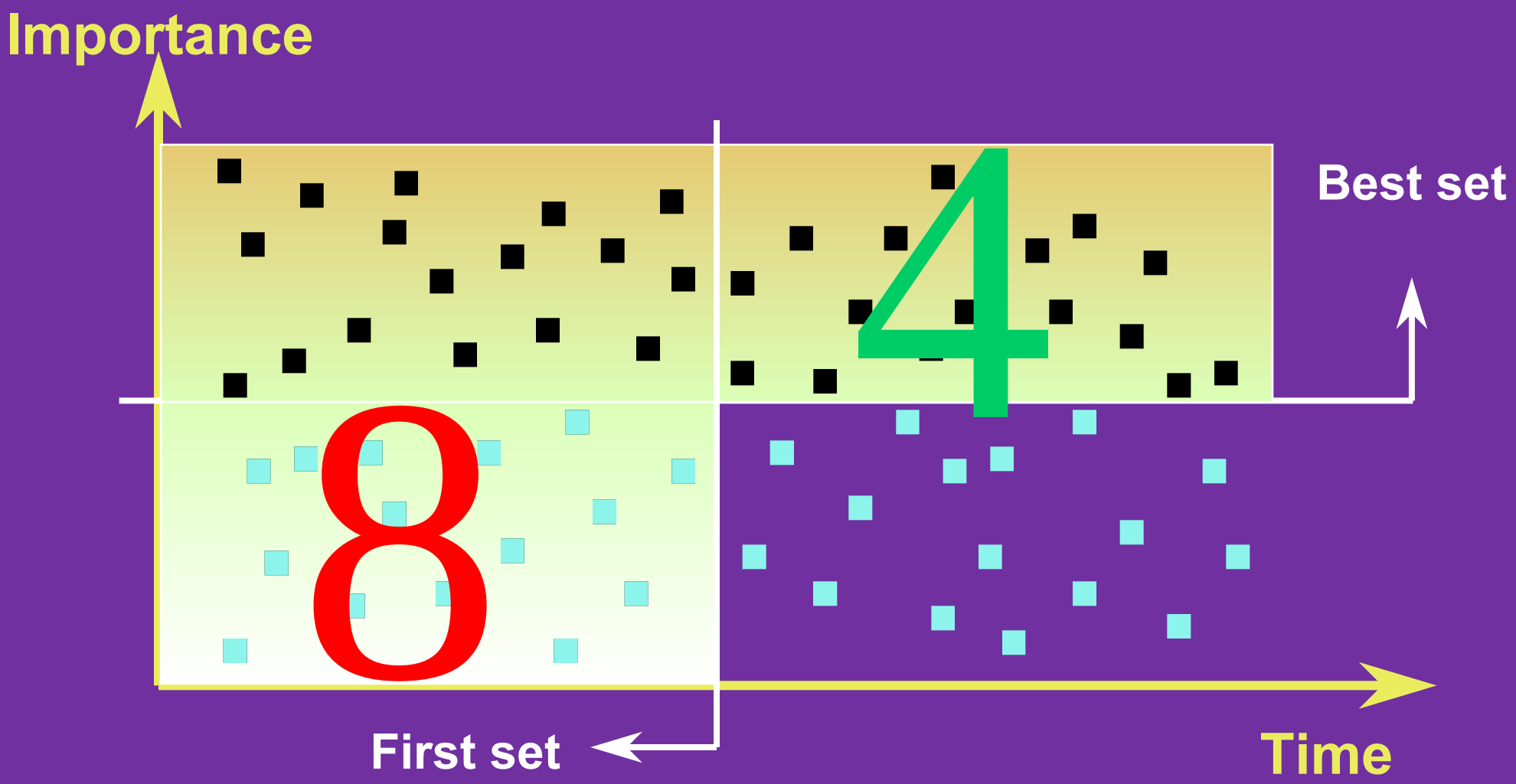
- **test specification can be broken down into three distinct tasks:**
  1. identify: determine ‘what’ is to be tested (identify test conditions) and prioritise
  2. design: determine ‘how’ the ‘what’ is to be tested (i.e. design test cases)
  3. build: implement the tests (data, scripts, etc.)

Task 1: identify conditions

(determine 'what' is to be tested and prioritise)

- **list the conditions that we would like to test:**
  - use the test design techniques specified in the test plan
  - there may be many conditions for each system function or attribute
  - e.g.
    - "life assurance for a winter sportsman"
    - "number items ordered > 99"
    - "date = 29-Feb-2004"
- **prioritise the test conditions**
  - must ensure most important conditions are covered

Selecting test conditions

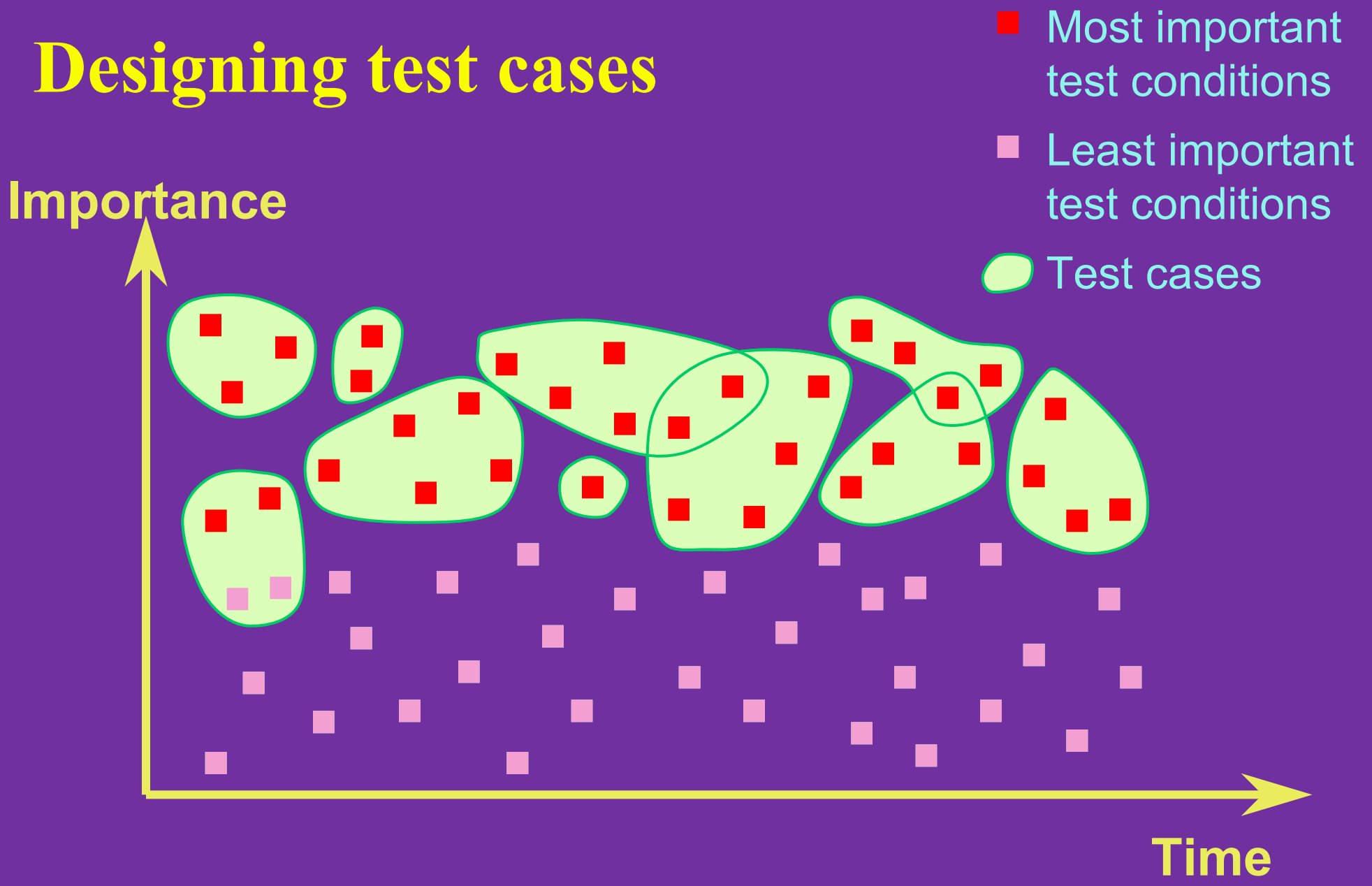


Task 2: design test cases

(determine 'how' the 'what' is to be tested)

- **design test input and test data**
  - each test exercises one or more test conditions
- **determine expected results**
  - predict the outcome of each test case, what is output, what is changed and what is not changed
- **design sets of tests**
  - different test sets for different objectives such as regression, building confidence, and finding faults

Designing test cases

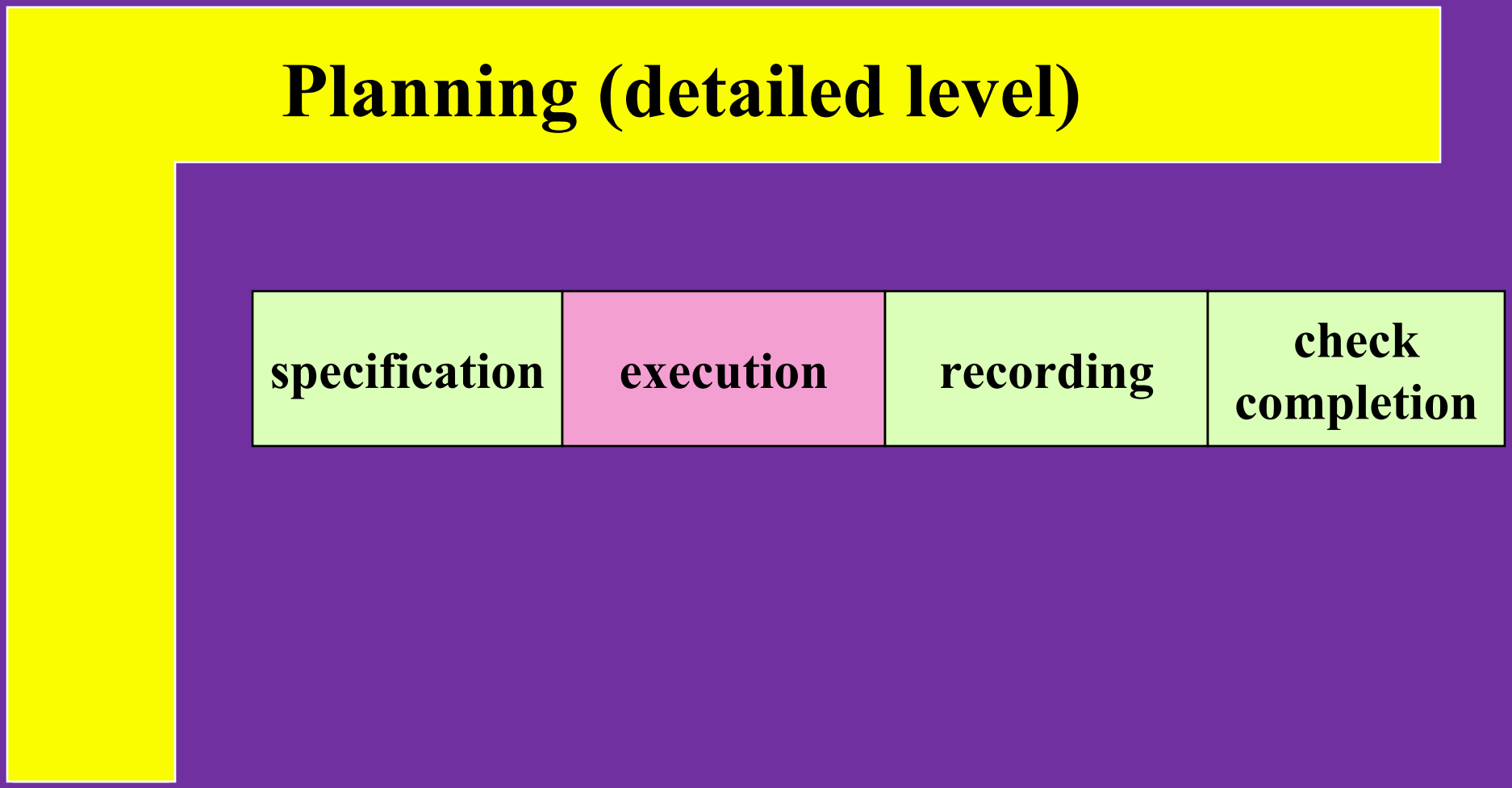


Task 3: build test cases

(implement the test cases)

- **prepare test scripts**
  - less system knowledge tester has the more detailed the scripts will have to be
  - scripts for tools have to specify every detail
- **prepare test data**
  - data that must exist in files and databases at the start of the tests
- **prepare expected results**
  - should be defined before the test is executed

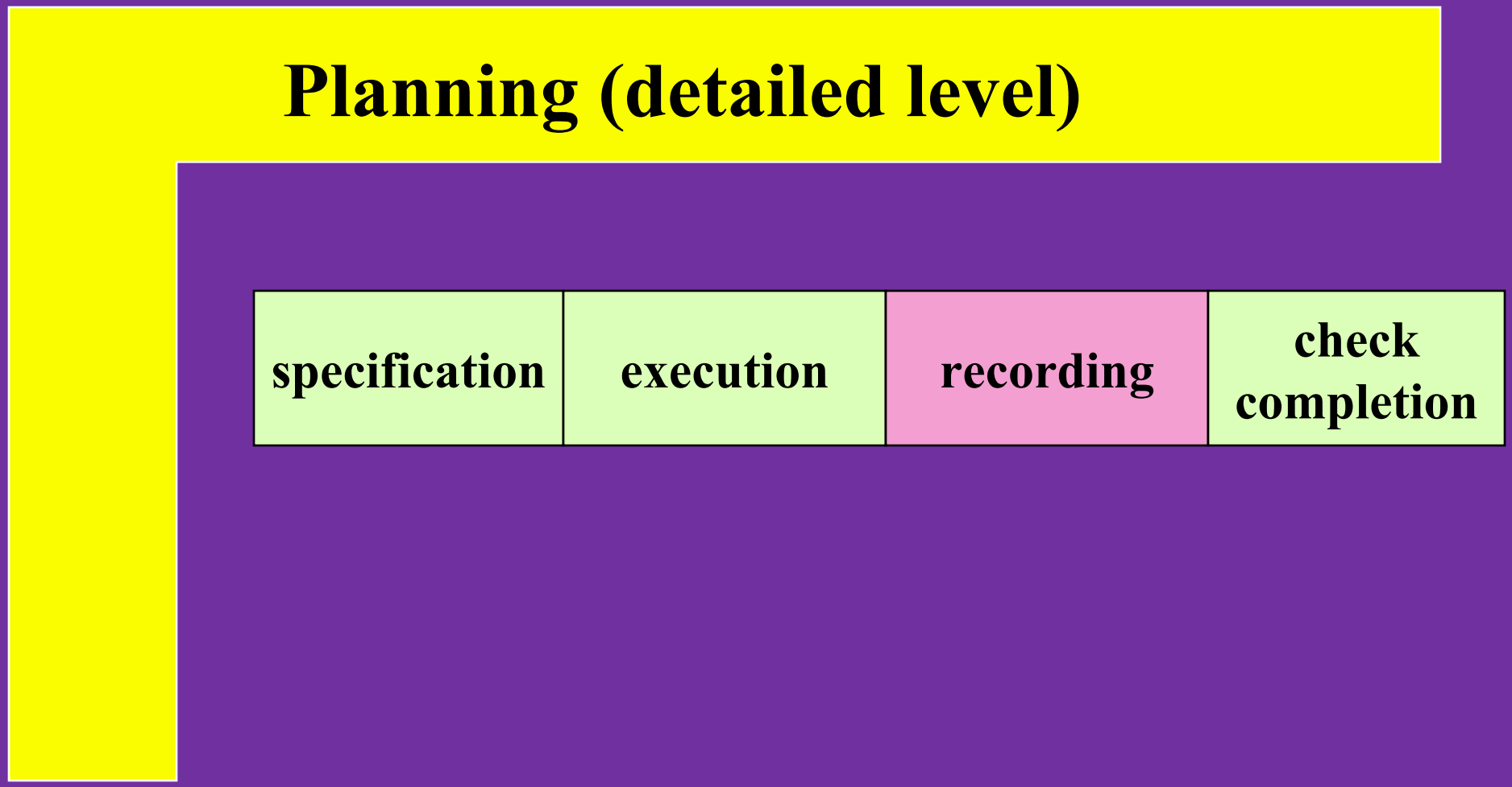
Test execution



Execution

- **Execute prescribed test cases**
  - most important ones first
  - would not execute all test cases if
    - testing only fault fixes
    - too many faults found by early test cases
    - time pressure
  - can be performed manually or automated

Test recording



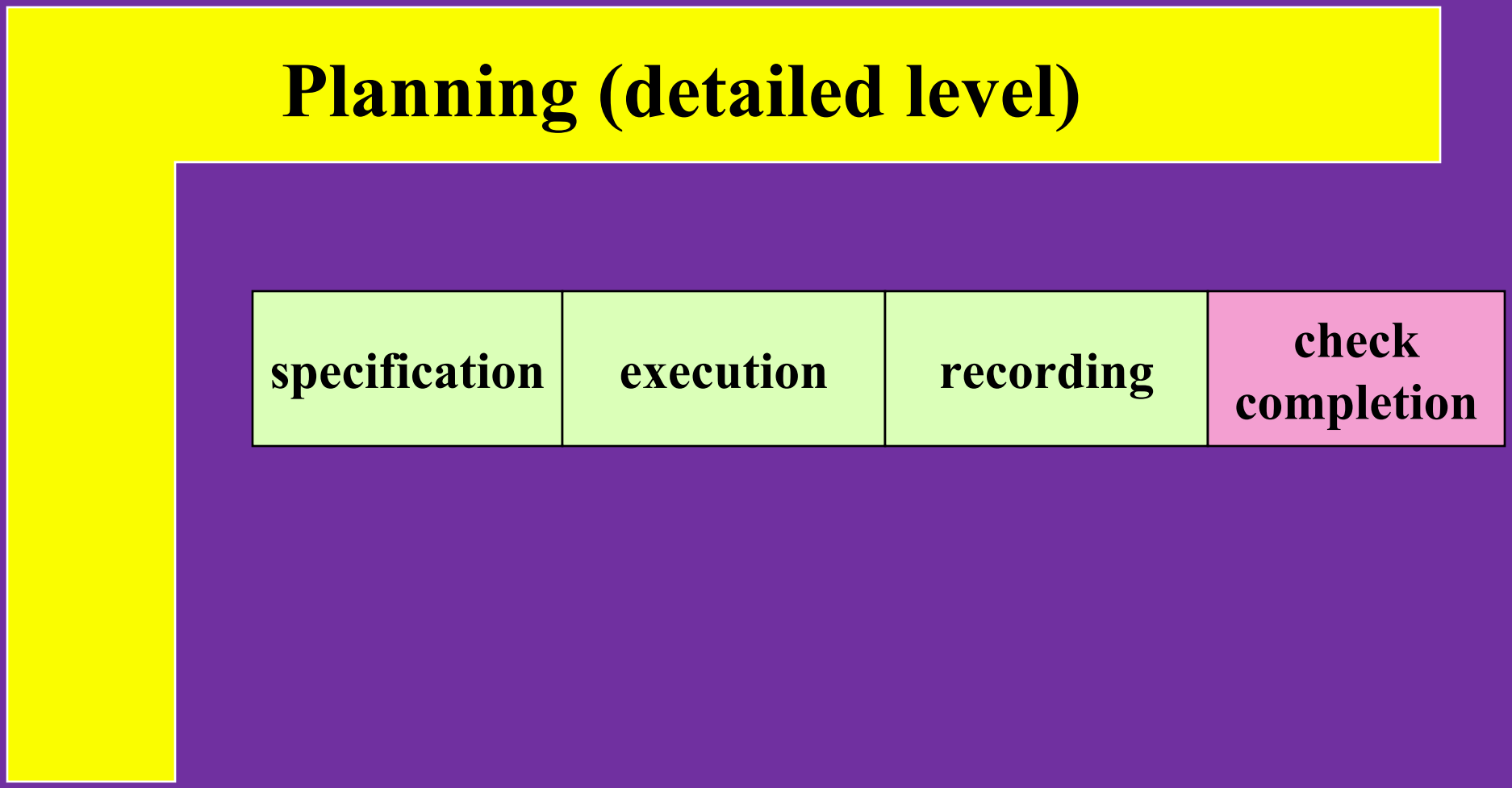
Test recording 1

- **The test record contains:**
  - identities and versions (unambiguously) of
    - software under test
    - test specifications
- **Follow the plan**
  - mark off progress on test script
  - document actual outcomes from the test
  - capture any other ideas you have for new test cases
  - note that these records are used to establish that all test activities have been carried out as specified

Test recording 2

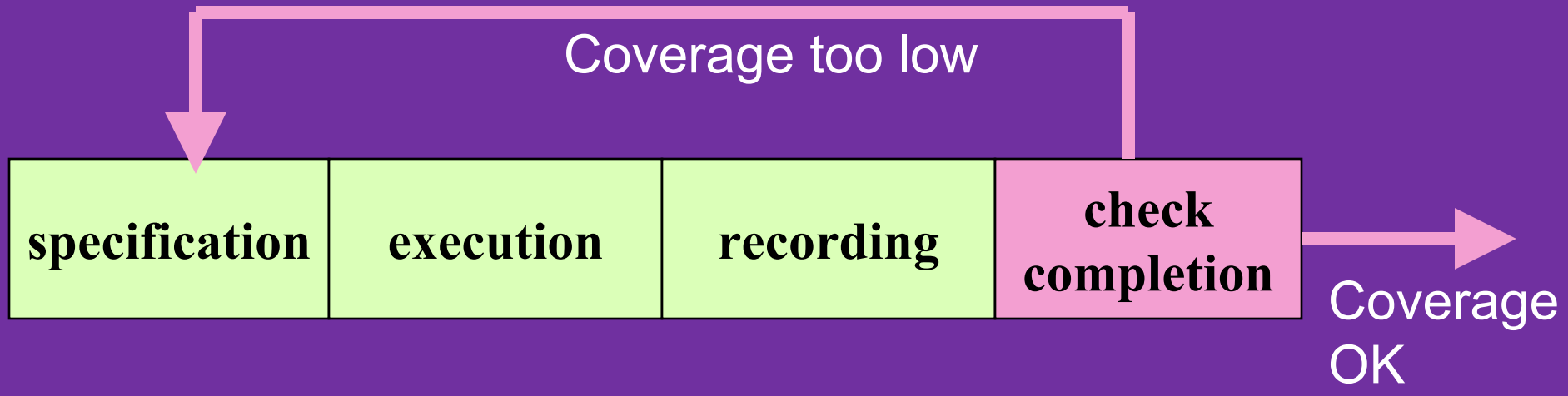
- **Compare actual outcome with expected outcome. Log discrepancies accordingly:**
  - software fault
  - test fault (e.g. expected results wrong)
  - environment or version fault
  - test run incorrectly
- **Log coverage levels achieved (for measures specified as test completion criteria)**
- **After the fault has been fixed, repeat the required test activities (execute, design, plan)**

Check test completion



Check test completion

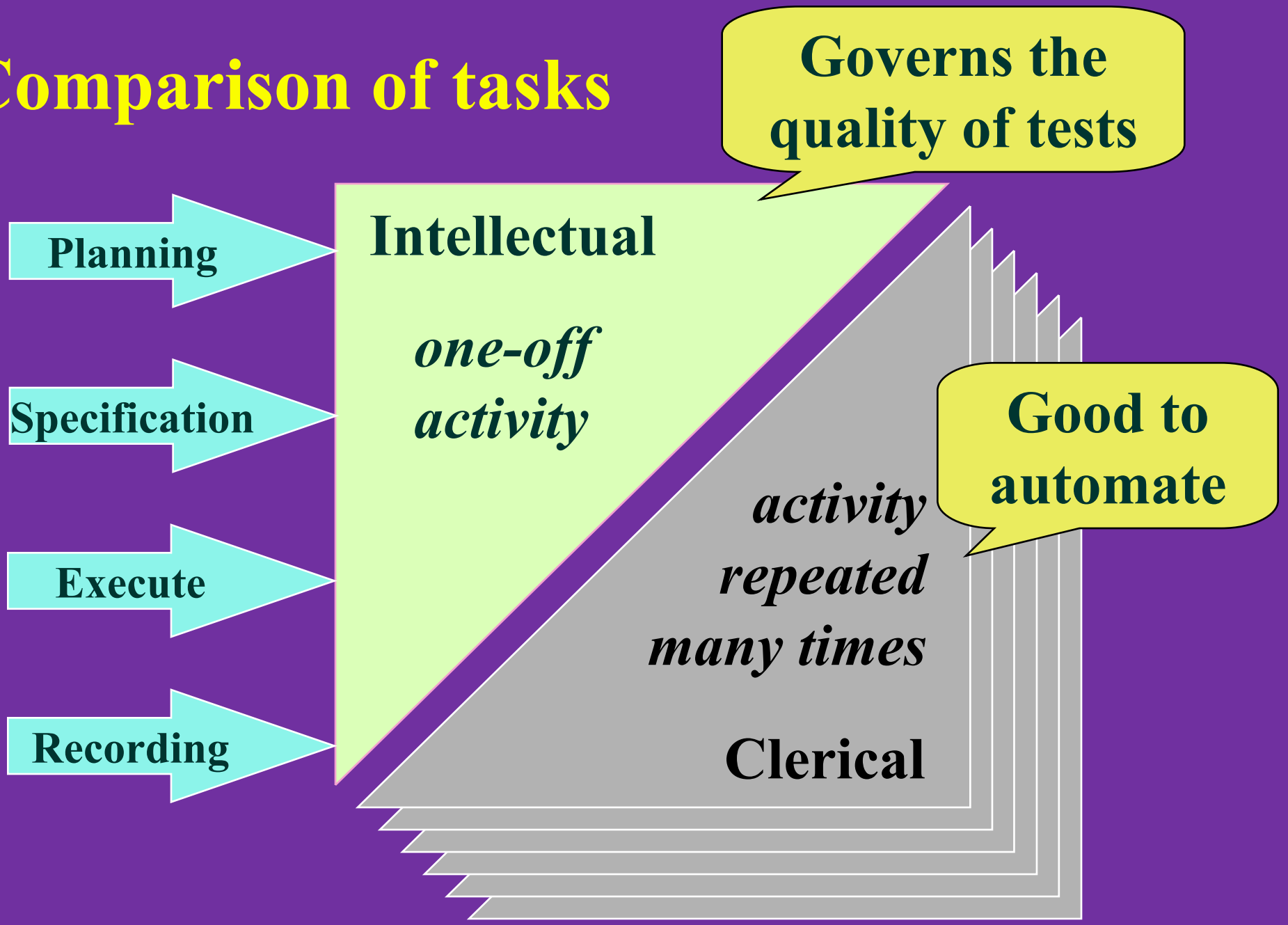
- Test completion criteria were specified in the test plan
- If not met, need to repeat test activities, e.g. test specification to design more tests



Test completion criteria

- **Completion or exit criteria apply to all levels of testing - to determine when to stop**
  - coverage, using a measurement technique, e.g.
    - branch coverage for unit testing
    - user requirements
    - most frequently used transactions
  - faults found (e.g. versus expected)
  - cost or time

Comparison of tasks



Principles

1	2	3
4	5	6

ISTQB / ISEB Foundation Exam Practice

Contents

Why testing is necessary

What is testing

Testing principles

Fundamental test process

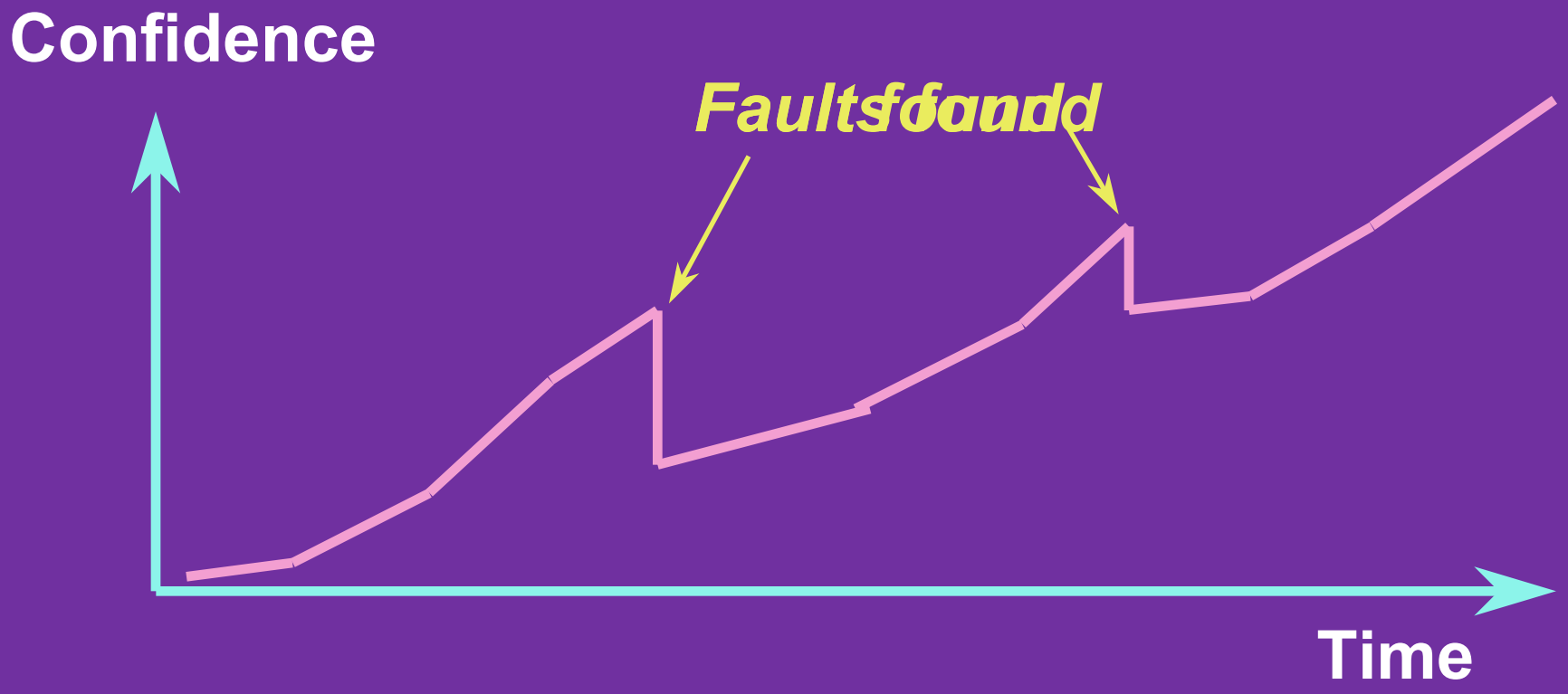
Psychology of testing

Code of Ethics

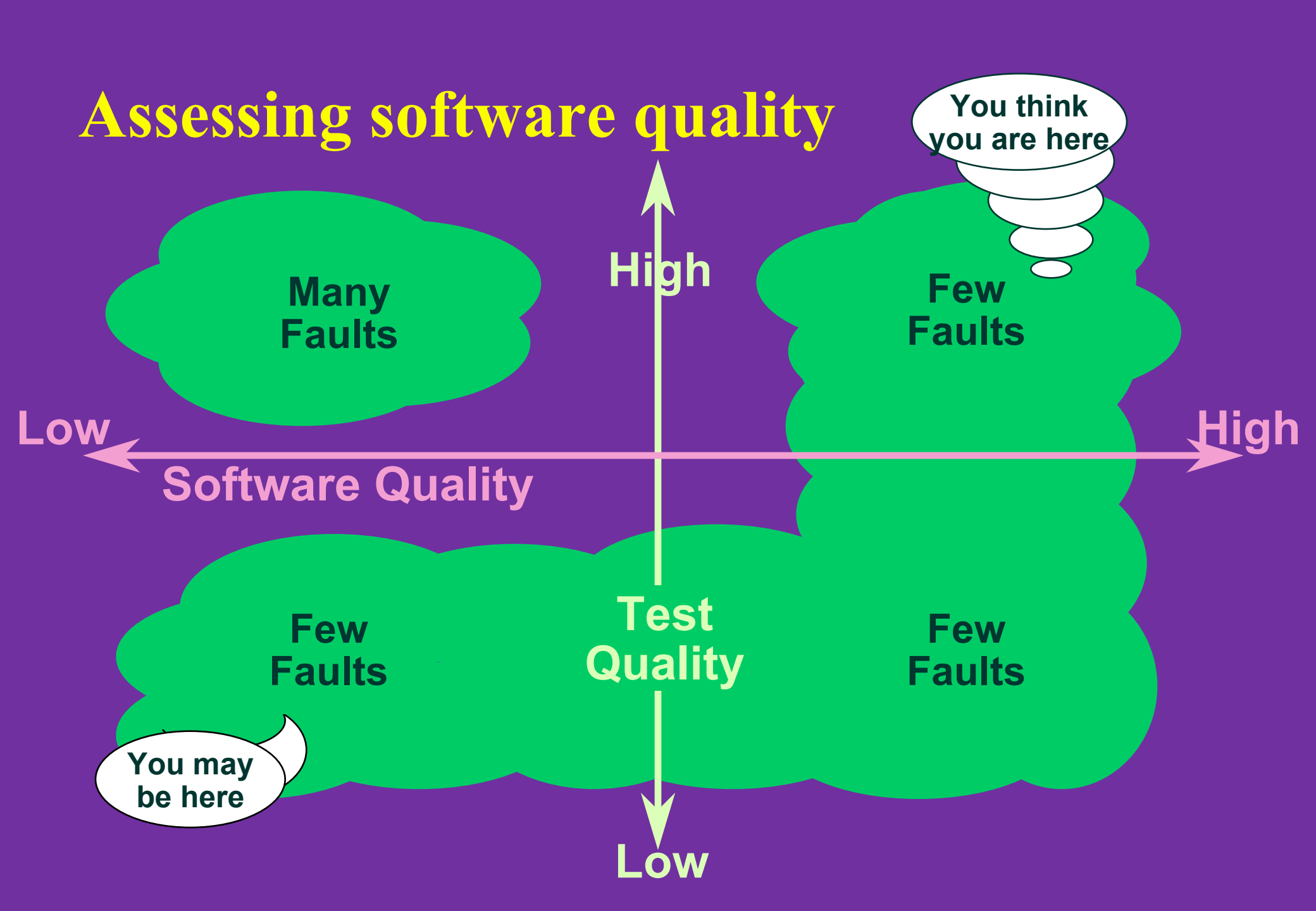
Why test?

- build confidence ✓
- prove that the software is correct ✗
- demonstrate conformance to requirements ✓
- find faults ✓
- reduce costs ✓
- show system meets user needs ✓
- assess the software quality ✓

Confidence



No faults found = confidence?


Assessing software quality
You think you are here
Many Faults
High
Few Faults
Low
High
Software Quality
Few Faults
Test Quality
Few Faults
You may be here
Low

A traditional testing approach

- **Show that the system:**
  - does what it should
  - doesn't do what it shouldn't

Goal:
show working

Success:
system works

Fastest achievement: easy test cases



Result: faults left in

A better testing approach

- **Show that the system:**
  - does what it shouldn't
  - doesn't do what it should

Goal:
find faults

Success:
system fails

Fastest achievement: difficult test cases



Result: fewer faults left in

The testing paradox

Purpose of testing: to find faults

Finding faults destroys confidence

∴ Purpose of testing: destroy confidence

Purpose of testing: build confidence

The best way to build confidence

is to try to destroy it

Who wants to be a tester?

- A destructive process
- Bring bad news ("your baby is ugly")
- Under worst time pressure (at the end)
- Need to take a different view, a different mindset ("What if it isn't?", "What could go wrong?")
- How should fault information be communicated (to authors and managers?)

Tester's have the right to:

- accurate information about progress and changes
- insight from developers about areas of the software
- delivered code tested to an agreed standard
- be regarded as a professional (no abuse!)
- find faults!
- challenge specifications and test plans
- have reported faults taken seriously (non-reproducible)
- make predictions about future fault levels
- improve your own testing process

Testers have responsibility to:

- follow the test plans, scripts etc. as documented
- report faults objectively and factually (no abuse!)
- check tests are correct before reporting s/w faults
- remember it is the software, not the programmer, that you are testing
- assess risk objectively
- prioritise what you report
- communicate the truth

Independence

- **Test your own work?**
  - find 30% - 50% of your own faults
  - same assumptions and thought processes
  - see what you meant or want to see, not what is there
  - emotional attachment
    - don’t want to find faults
    - actively want NOT to find faults

Levels of independence

- None: tests designed by the person who wrote the software
- Tests designed by a different person
- Tests designed by someone from a different department or team (e.g. test team)
- Tests designed by someone from a different organisation (e.g. agency)
- Tests generated by a tool (low quality tests?)

Principles

1	2	3
4	5	6

Contents

Why testing is necessary

What is testing

Testing principles

Fundamental test process

Psychology of testing

Code of Ethics

Code of Ethics

- ISTQB Code of Ethics:
  - PUBLIC
  - CLIENT AND EMPLOYER
  - PRODUCT
  - JUDGMENT
  - MANAGEMENT
  - PROFESSION
  - COLLEAGUES
  - SELF

Principles

1	2	3
4	5	6

Summary: Key Points

Why testing is necessary

What is testing

Testing principles

Fundamental test process

Psychology of testing

Code of Ethics

Chapter 2

Software Testing

ISTQB / ISEB Foundation Exam Practice

Testing Throughout the software Life Cycle

1 Principles	2 Lifecycle	3 Static testing
4 Test design techniques	5 Management	6 Tools

Lifecycle

1	2	3
4	5	6

ISTQB / ISEB Foundation Exam Practice

Contents

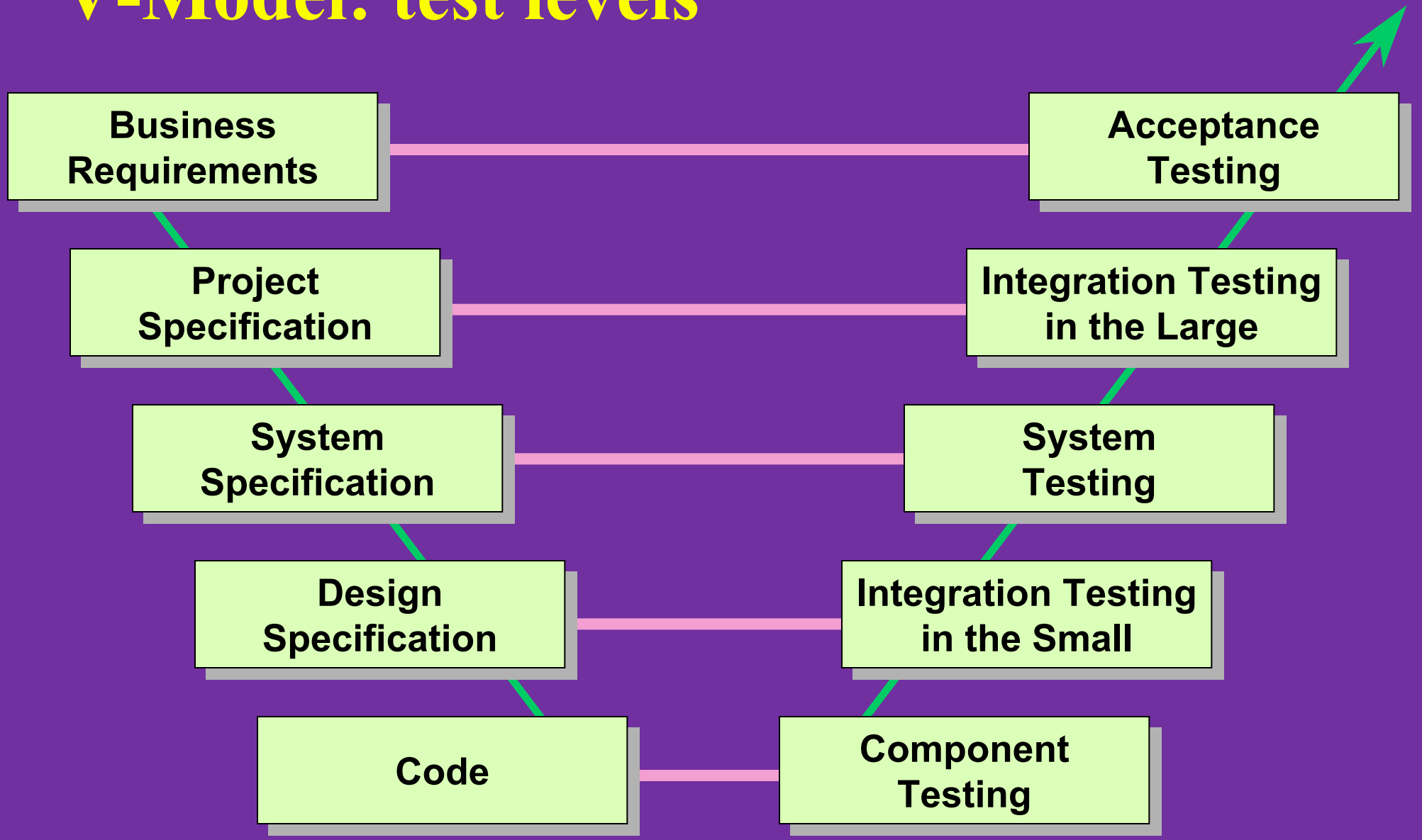
Software Models

Test level

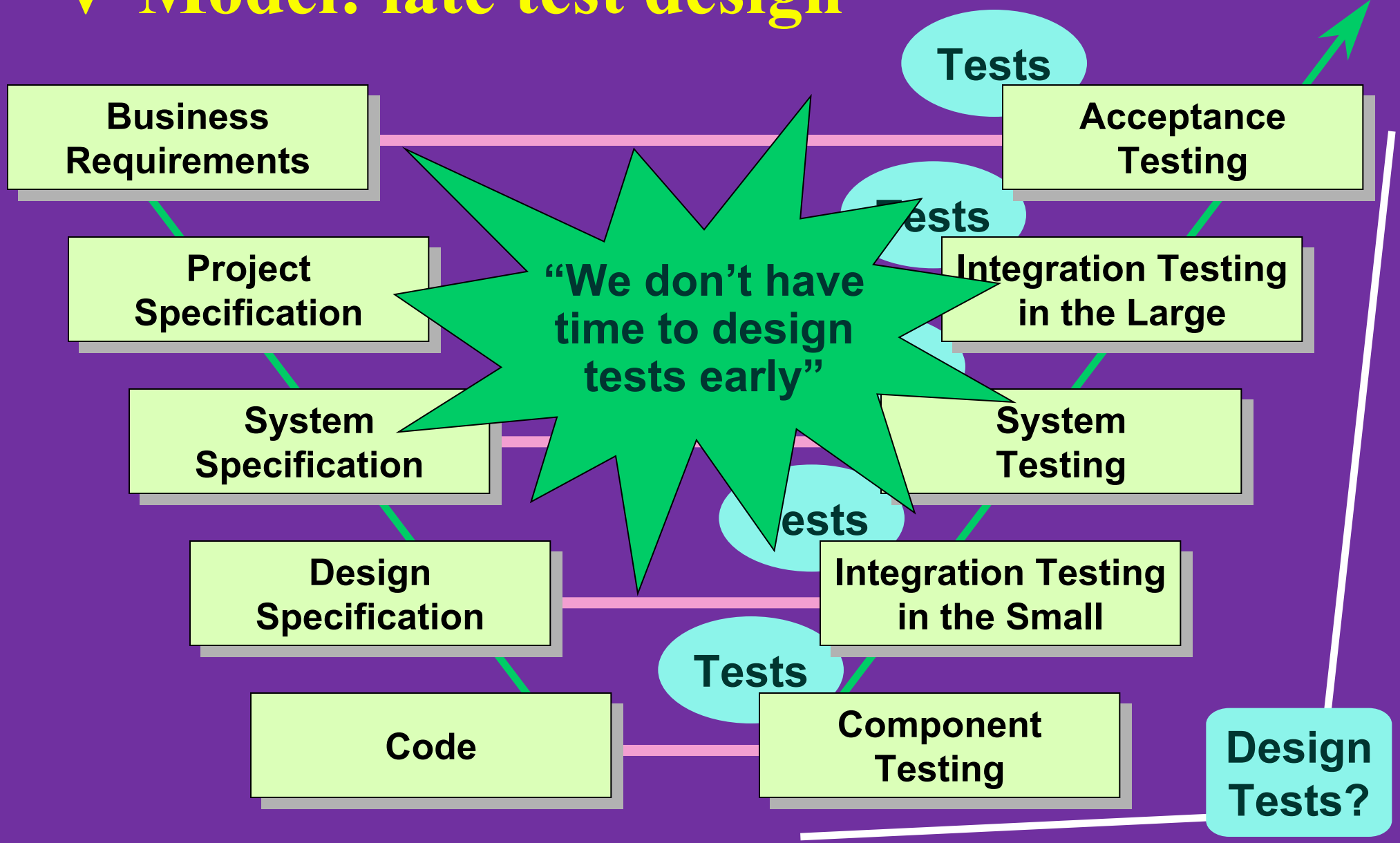
Test type

Maintenance testing

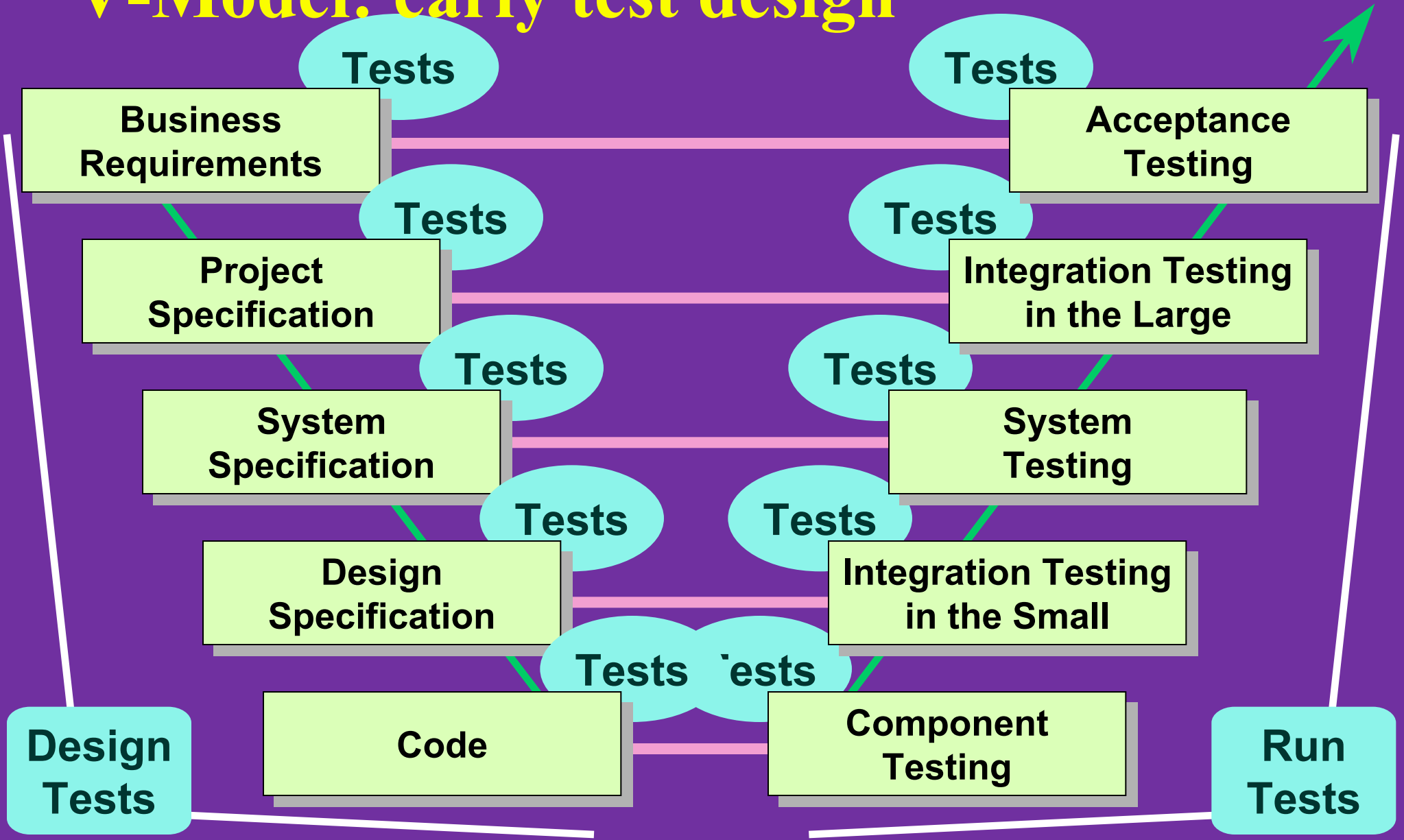
V-Model: test levels



V-Model: late test design



V-Model: early test design

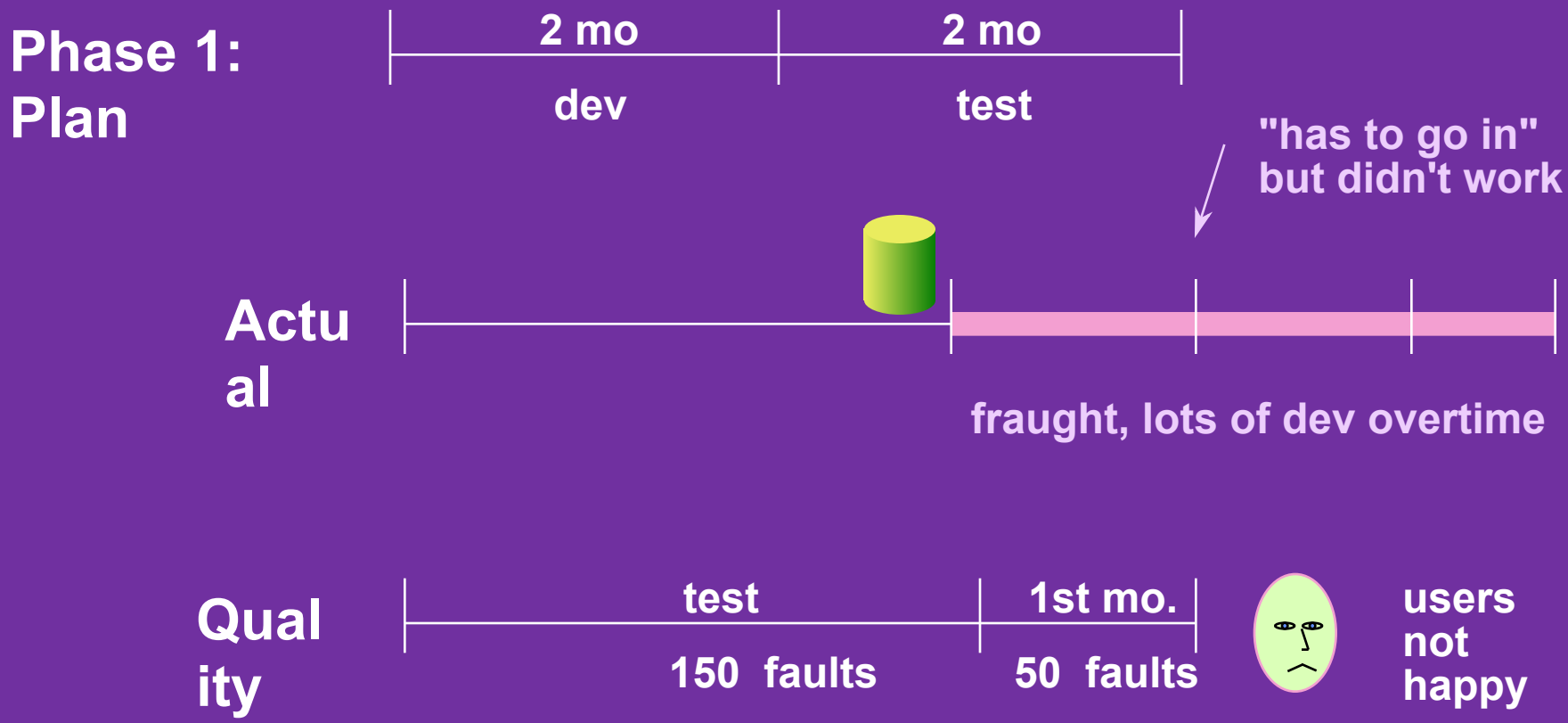


Early test design

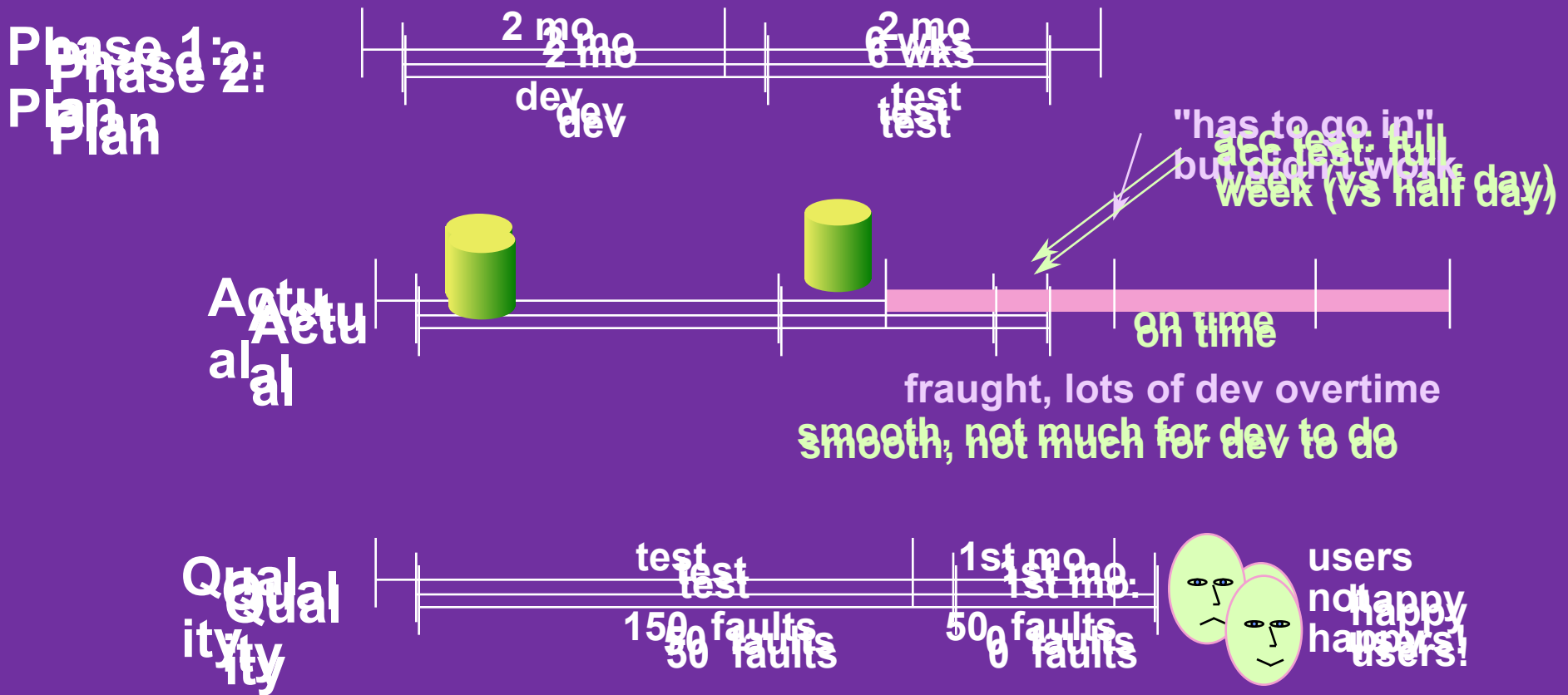
- test design finds faults
- faults found early are cheaper to fix
- most significant faults found first
- faults prevented, not built in
- no additional effort, re-schedule test design
- changing requirements caused by test design

Early test design helps to build quality, stops fault multiplication

Experience report: Phase 1



Experience report: Phase 2

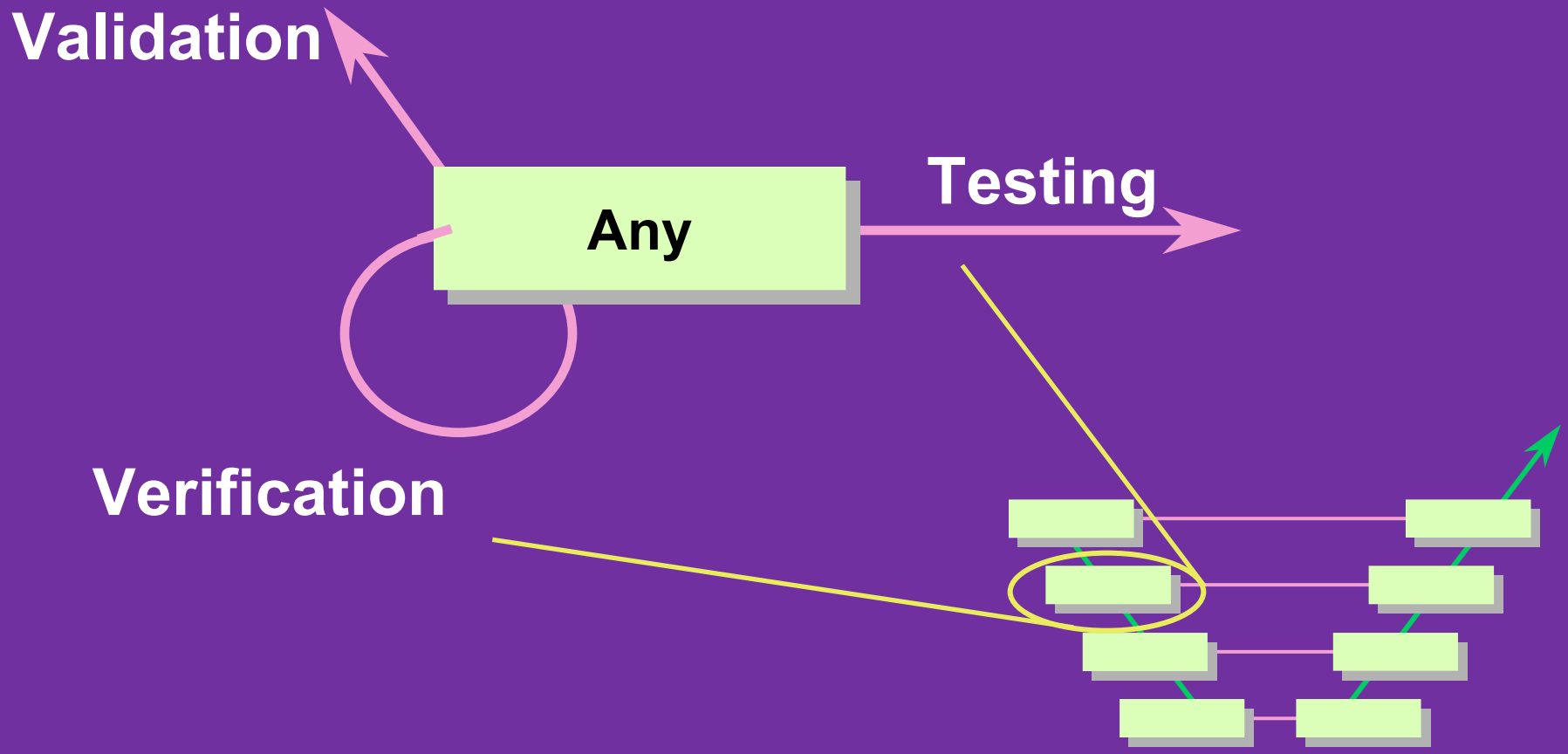


Source: Simon Barlow & Alan Veitch, Scottish Widows, Feb 96

VV&T

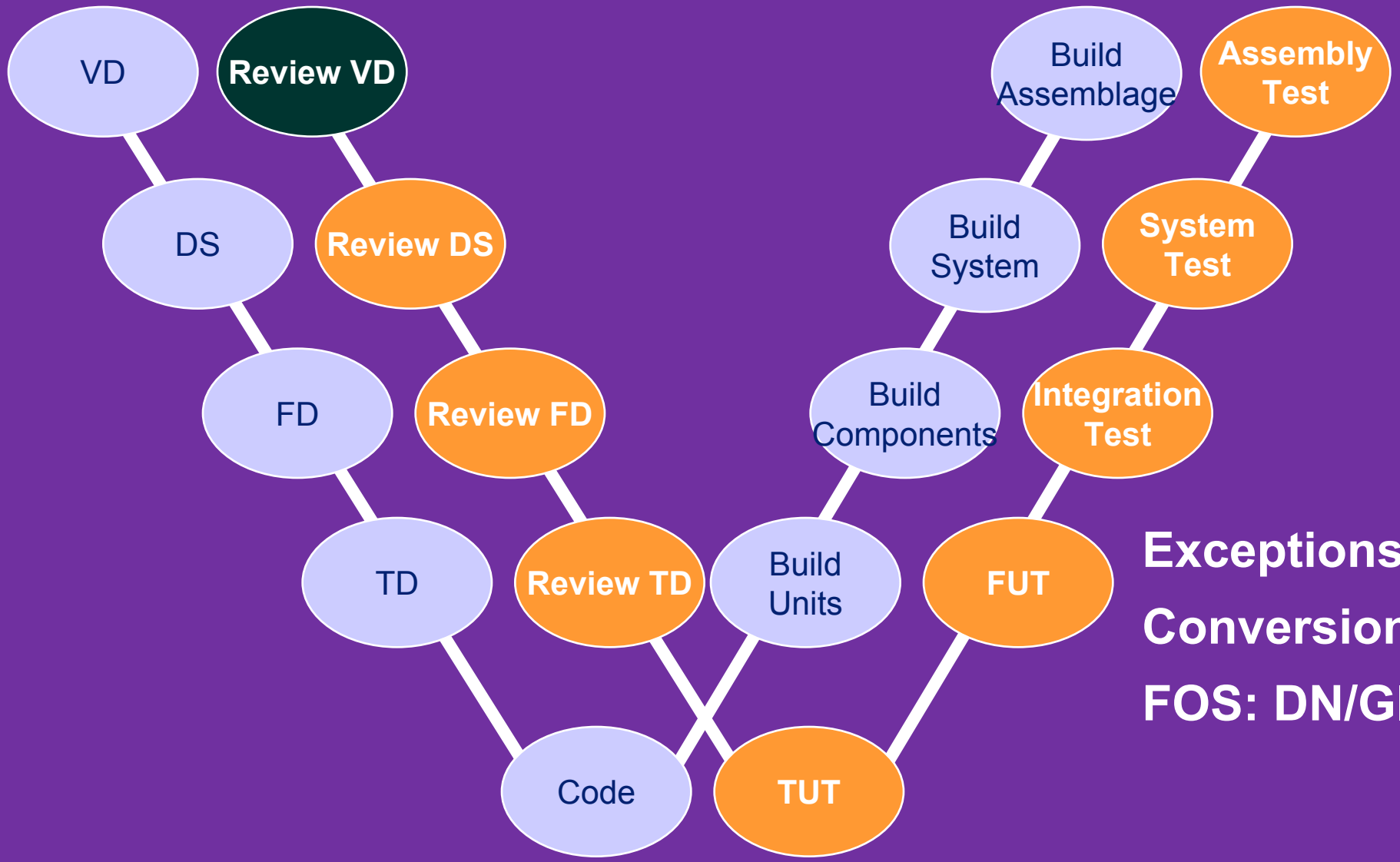
- **Verification**
  - the process of evaluating a system or component to determine whether the products of the given development phase satisfy the conditions imposed at the start of that phase [BS 7925-1]
- **Validation**
  - determination of the correctness of the products of software development with respect to the user needs and requirements [BS 7925-1]
- **Testing**
  - the process of exercising software to verify that it satisfies specified requirements and to detect faults

Verification, Validation and Testing



V-model exercise

The V Model - Exercise



Exceptions:

Conversion Test

FOS: DN/GIdn

How would you test this spec?

- A computer program plays chess with one user. It displays the board and the pieces on the screen. Moves are made by dragging pieces.

“Testing is expensive”

- **Compared to what?**
- **What is the cost of NOT testing, or of faults missed that should have been found in test?**
  - Cost to fix faults escalates the later the fault is found
  - Poor quality software costs more to use
    - users take more time to understand what to do
    - users make more mistakes in using it
    - morale suffers
    - => lower productivity
- **Do you know what it costs your organisation?**

What do software faults cost?

- **Have you ever accidentally destroyed a PC?**
  - knocked it off your desk?
  - poured coffee into the hard disc drive?
  - dropped it out of a 2nd storey window?
- **How would you feel?**
- **How much would it cost?**

Hypothetical Cost - 1

(Loaded Salary cost: £50/hr)

Fault Cost	Developer	User
- detect (.5 hr)		£25
- report (.5 hr)		£25
- receive & process (1 hr)	£50	
- assign & bkgnd (4 hrs)	£200	
- debug (.5 hr)	£25	
- test fault fix (.5 hr)	£25	
- regression test (8 hrs)	£400	
	£700	£50

Hypothetical Cost - 2

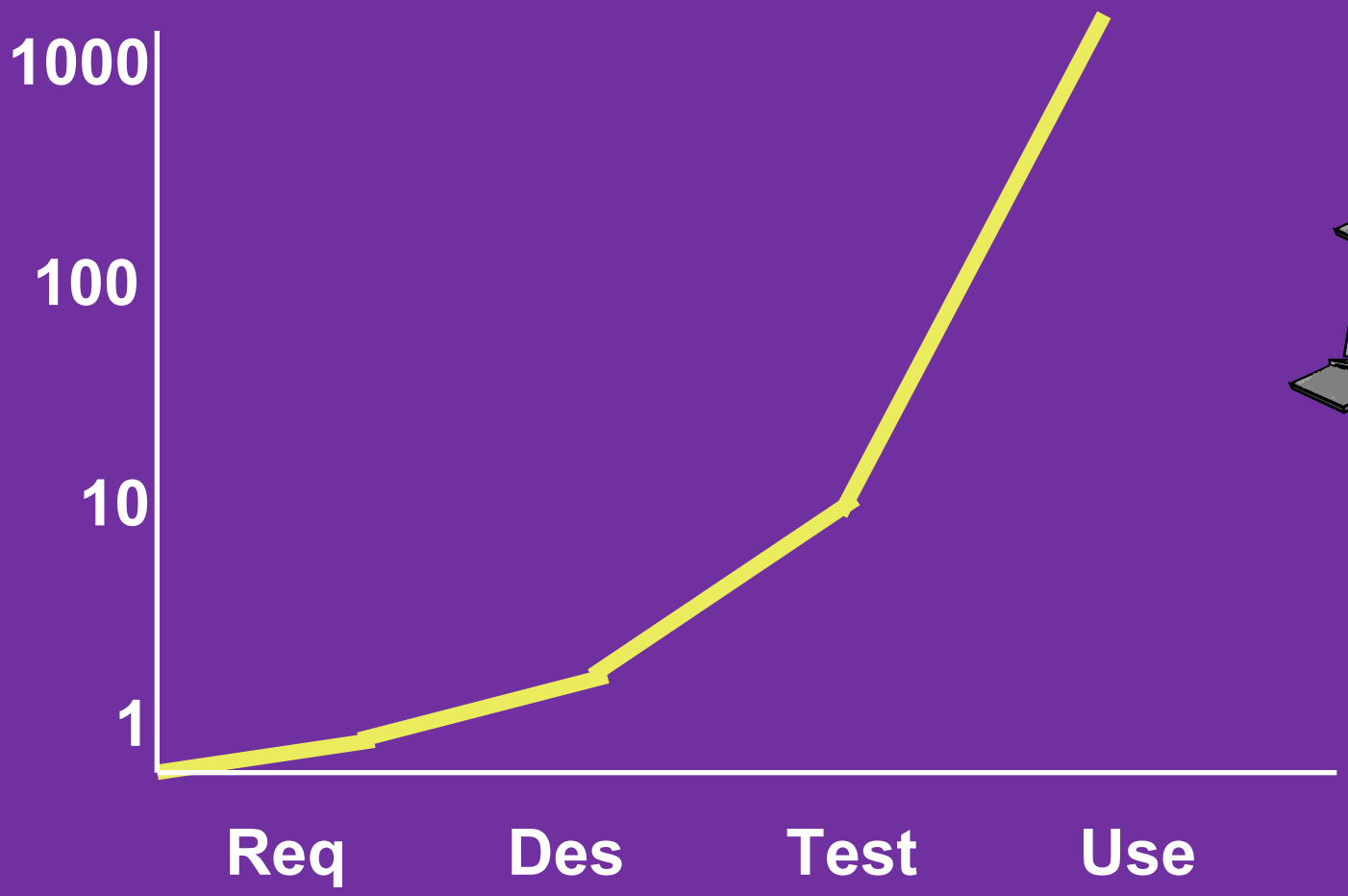
Fault Cost	Developer	User
	£700	£50
- update doc'n, CM (2 hrs)	£100	
- update code library (1 hr)	£50	
- inform users (1 hr)	£50	
- admin(10% = 2 hrs)	£100	
Total (20 hrs)	£1000	

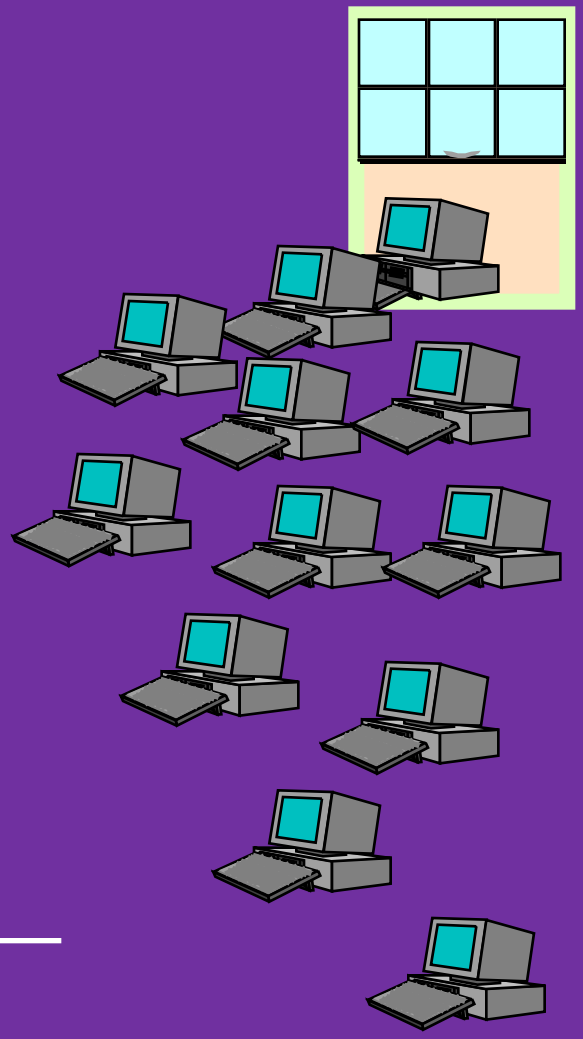


Hypothetical Cost - 3

Fault Cost	Developer	User
	£1000	£50
(suppose affects only 5 users)		
- work x 2, 1 wk		£4000
- fix data (1 day)		£350
- pay for fix (3 days maint)		£750
- regr test & sign off (2 days)		£700
- update doc'n / inform (1 day)		£350
- double check + 12% 5 wks		£5000
- admin (+7.5%)		£800
Totals	£1000	£12000

Cost of fixing faults





How expensive for you?

- **Do your own calculation**
  - calculate cost of testing
    - people's time, machines, tools
  - calculate cost to fix faults found in testing
  - calculate cost to fix faults missed by testing
- **Estimate if no data available**
  - your figures will be the best your company has!

(10 minutes)

Lifecycle

1	2	3
4	5	6

ISTQB / ISEB Foundation Exam Practice

Contents

Models for testing, economics of testing

Test level

Test type

Maintenance testing

(Before planning for a set of tests)

- set organisational test strategy
- identify people to be involved (sponsors, testers, QA, development, support, et al.)
- examine the requirements or functional specifications (test basis)
- set up the test organisation and infrastructure
- defining test deliverables & reporting structure

See: Structured Testing, an introduction to TMap®, Pol & van Veenendaal, 1998

High level test planning

- **What is the purpose of a high level test plan?**
  - Who does it communicate to?
  - Why is it a good idea to have one?
- **What information should be in a high level test plan?**
  - What is your standard for contents of a test plan?
  - Have you ever forgotten something important?
  - What is not included in a test plan?

Test Plan 1

- **1 Test Plan Identifier**
- **2 Introduction**
  - software items and features to be tested
  - references to project authorisation, project plan, QA plan, CM plan, relevant policies & standards
- **3 Test items**
  - test items including version/revision level
  - how transmitted (net, disc, CD, etc.)
  - references to software documentation

Source: ANSI/IEEE Std 829-1998, Test Documentation

Test Plan 2

- **4 Features to be tested**
  - identify test design specification / techniques
- **5 Features not to be tested**
  - reasons for exclusion

Test Plan 3

- **6 Approach**
  - activities, techniques and tools
  - detailed enough to estimate
  - specify degree of comprehensiveness (e.g. coverage) and other completion criteria (e.g. faults)
  - identify constraints (environment, staff, deadlines)
- **7 Item Pass/Fail Criteria**
- **8 Suspension criteria and resumption criteria**
  - for all or parts of testing activities
  - which activities must be repeated on resumption

Test Plan 4

- **9 Test Deliverables**
  - Test plan
  - Test design specification
  - Test case specification
  - Test procedure specification
  - Test item transmittal reports
  - Test logs
  - Test incident reports
  - Test summary reports

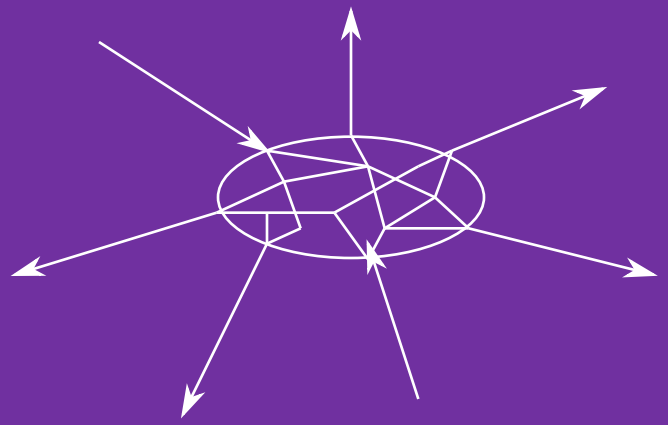
Test Plan 5

- **10 Testing tasks**
  - including inter-task dependencies & special skills
- **11 Environment**
  - physical, hardware, software, tools
  - mode of usage, security, office space
- **12 Responsibilities**
  - to manage, design, prepare, execute, witness, check, resolve issues, providing environment, providing the software to test

Test Plan 6

- **13 Staffing and Training Needs**
- **14 Schedule**
  - test milestones in project schedule
  - item transmittal milestones
  - additional test milestones (environment ready)
  - what resources are needed when
- **15 Risks and Contingencies**
  - contingency plan for each identified risk
- **16 Approvals**
  - names and when approved

Component testing



- lowest level
- tested in isolation
- most thorough look at detail
  - error handling
  - interfaces
- usually done by programmer
- also known as unit, module, program testing

Component test strategy 1

- **specify test design techniques and rationale**
  - from Section 3 of the standard*
- **specify criteria for test completion and rationale**
  - from Section 4 of the standard
- **document the degree of independence for test design**
  - component author, another person, from different section, from different organisation, non-human

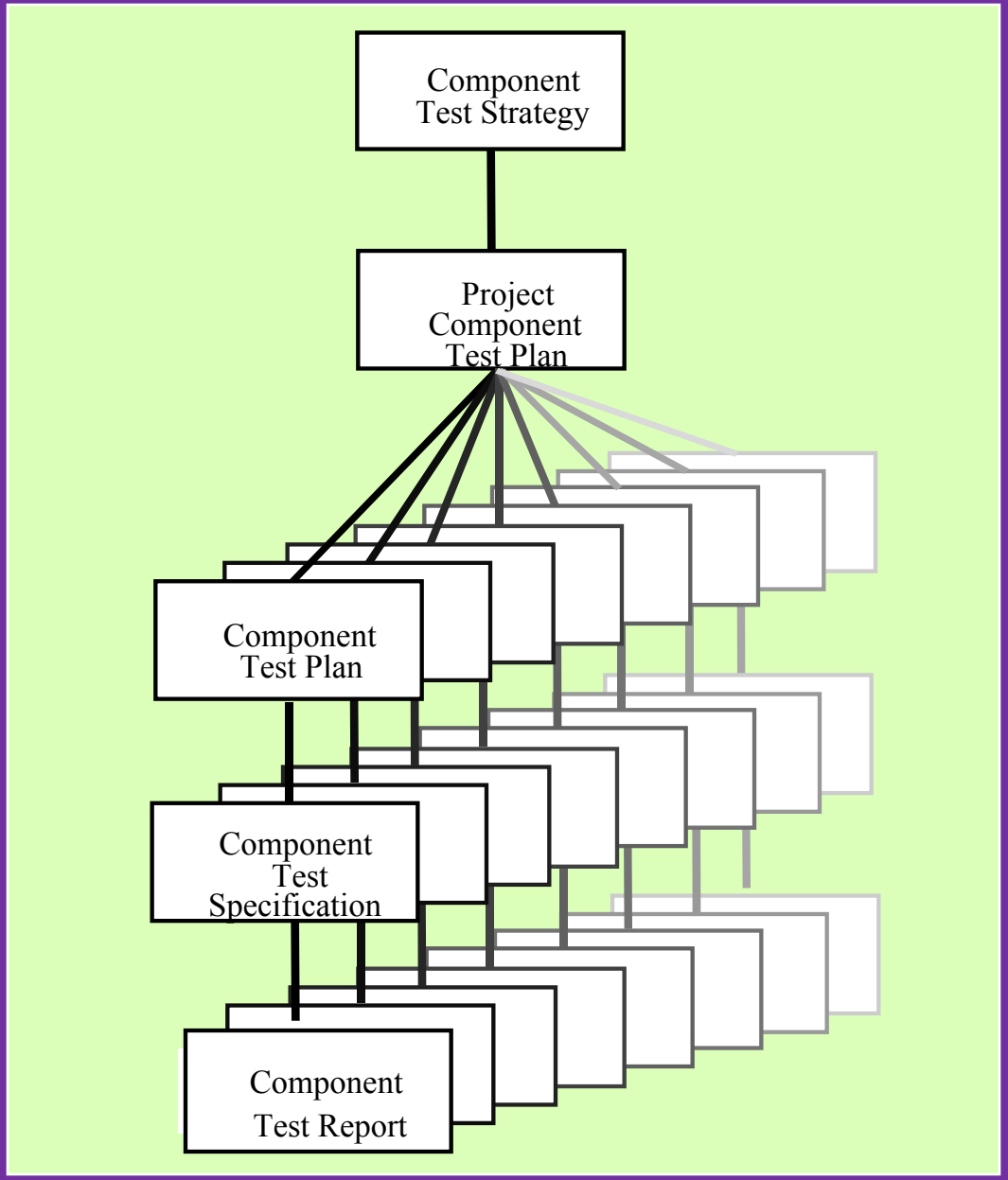
*Source: BS 7925-2, Software Component Testing Standard

Component test strategy 2

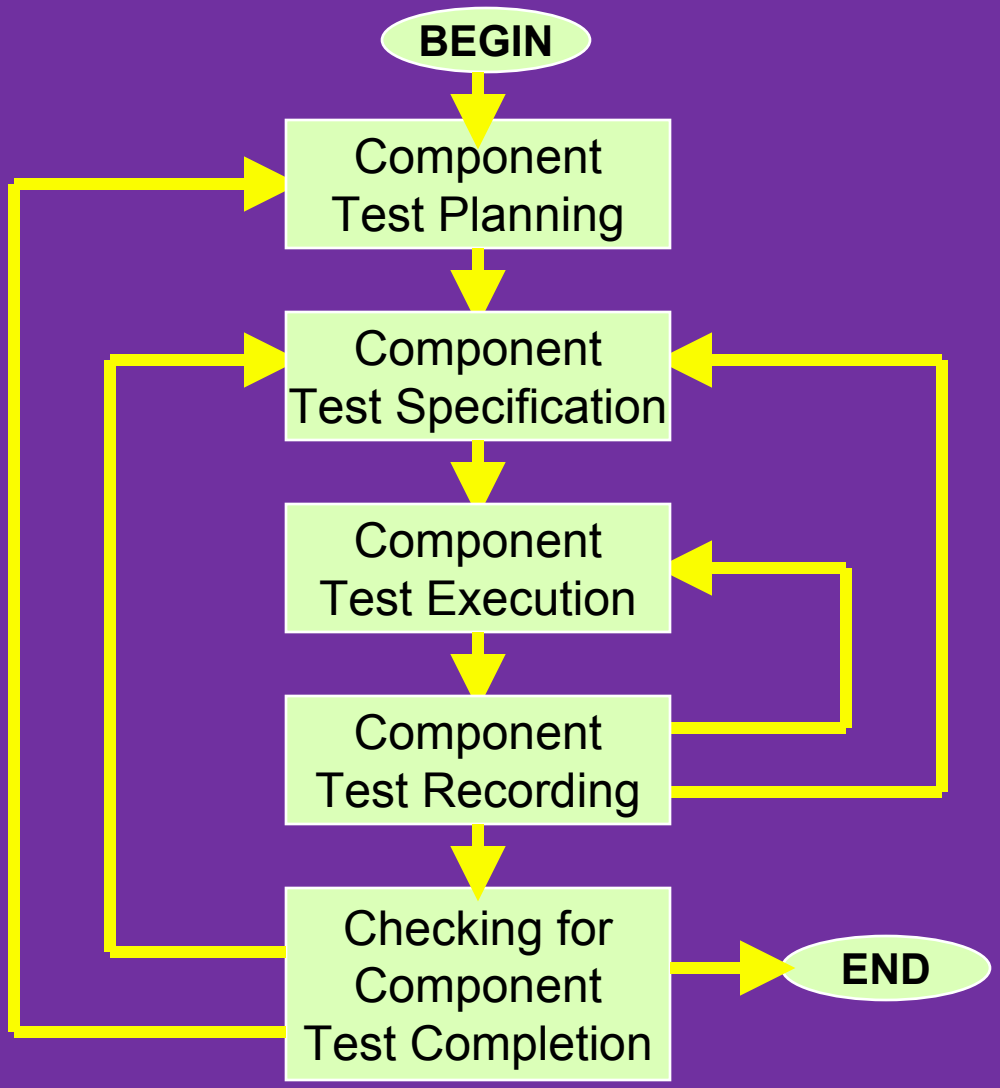
- **component integration and environment**
  - isolation, top-down, bottom-up, or mixture
  - hardware and software
- **document test process and activities**
  - including inputs and outputs of each activity
- **affected activities are repeated after any fault fixes or changes**
- **project component test plan**
  - dependencies between component tests

Component Test Document Hierarchy

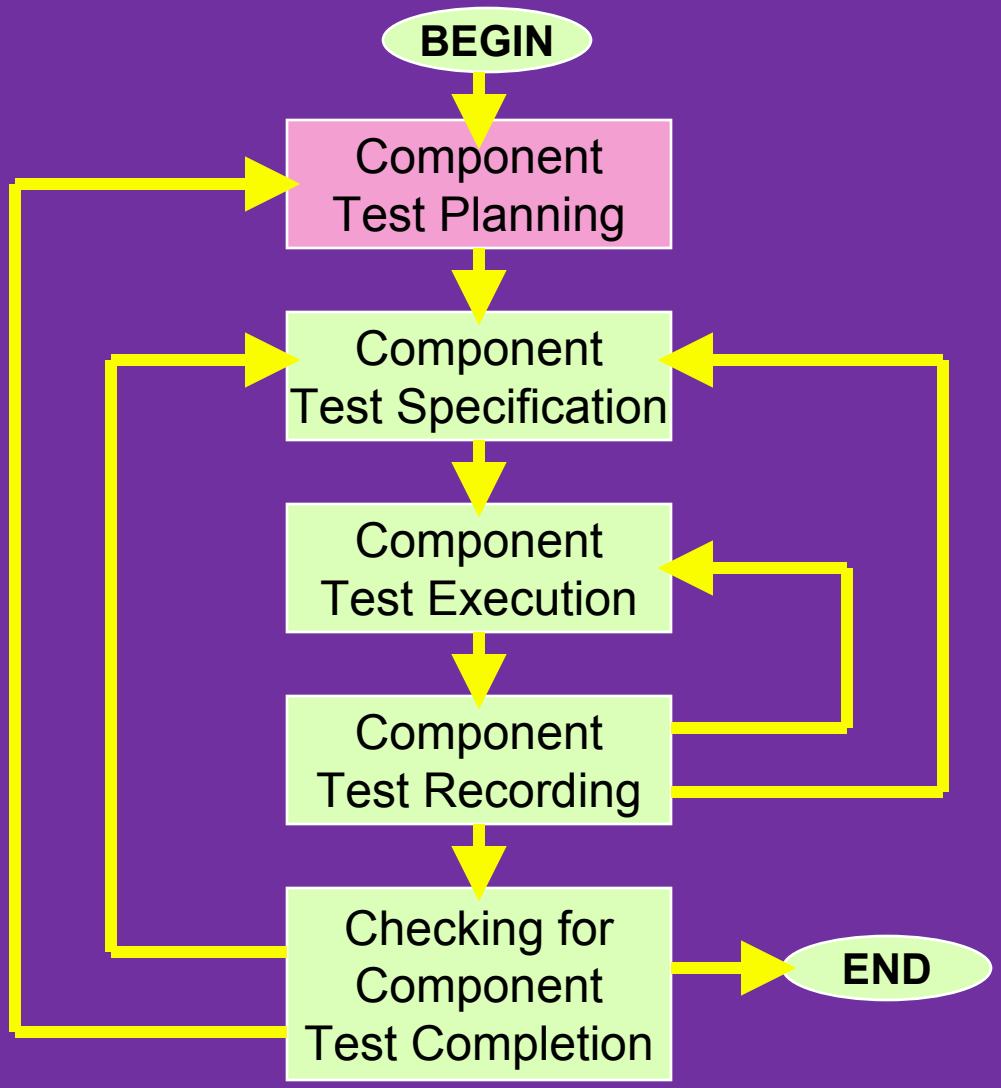
Source: BS 7925-2, Software Component Testing Standard, Annex A



Component test process



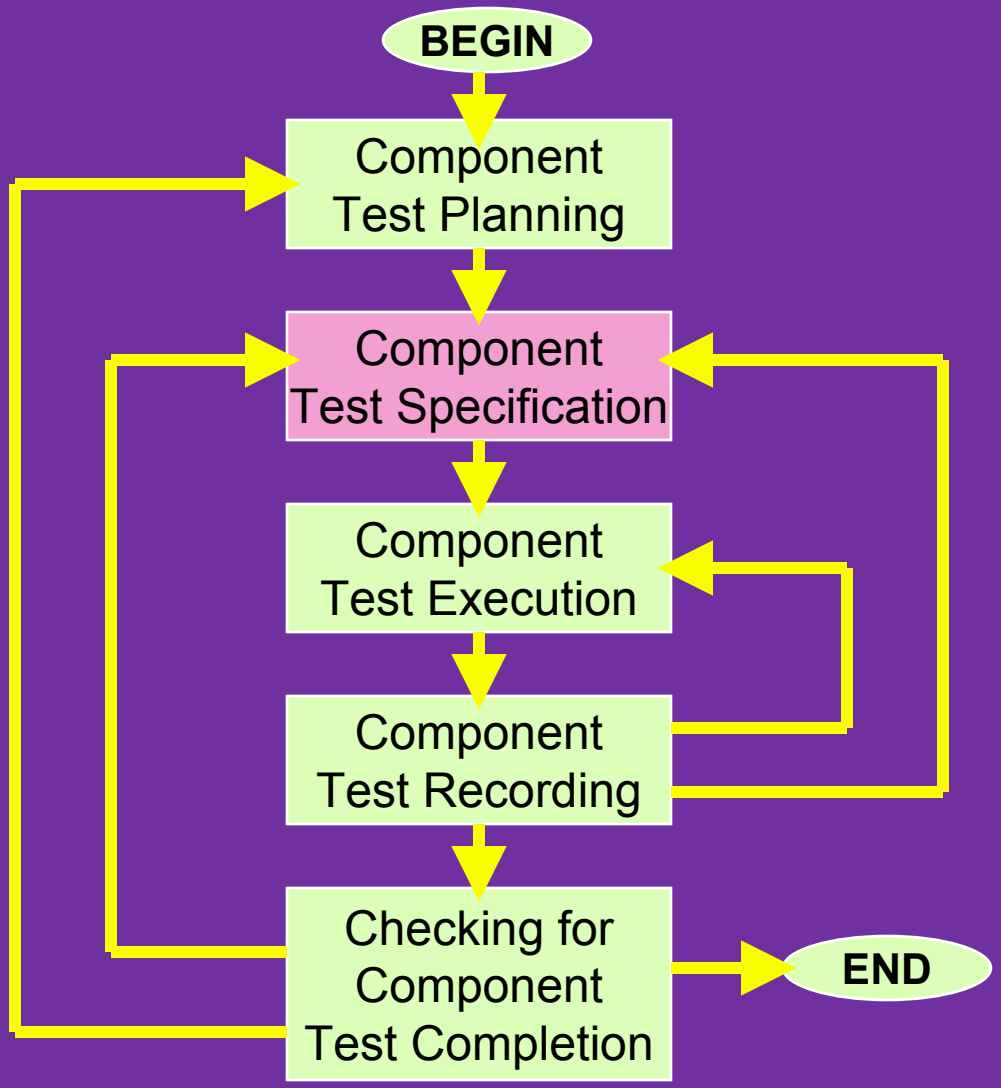
Component test process



Component test planning

- how the test strategy and project test plan apply to the component under test
- any exceptions to the strategy
- all software the component will interact with (e.g. stubs and drivers

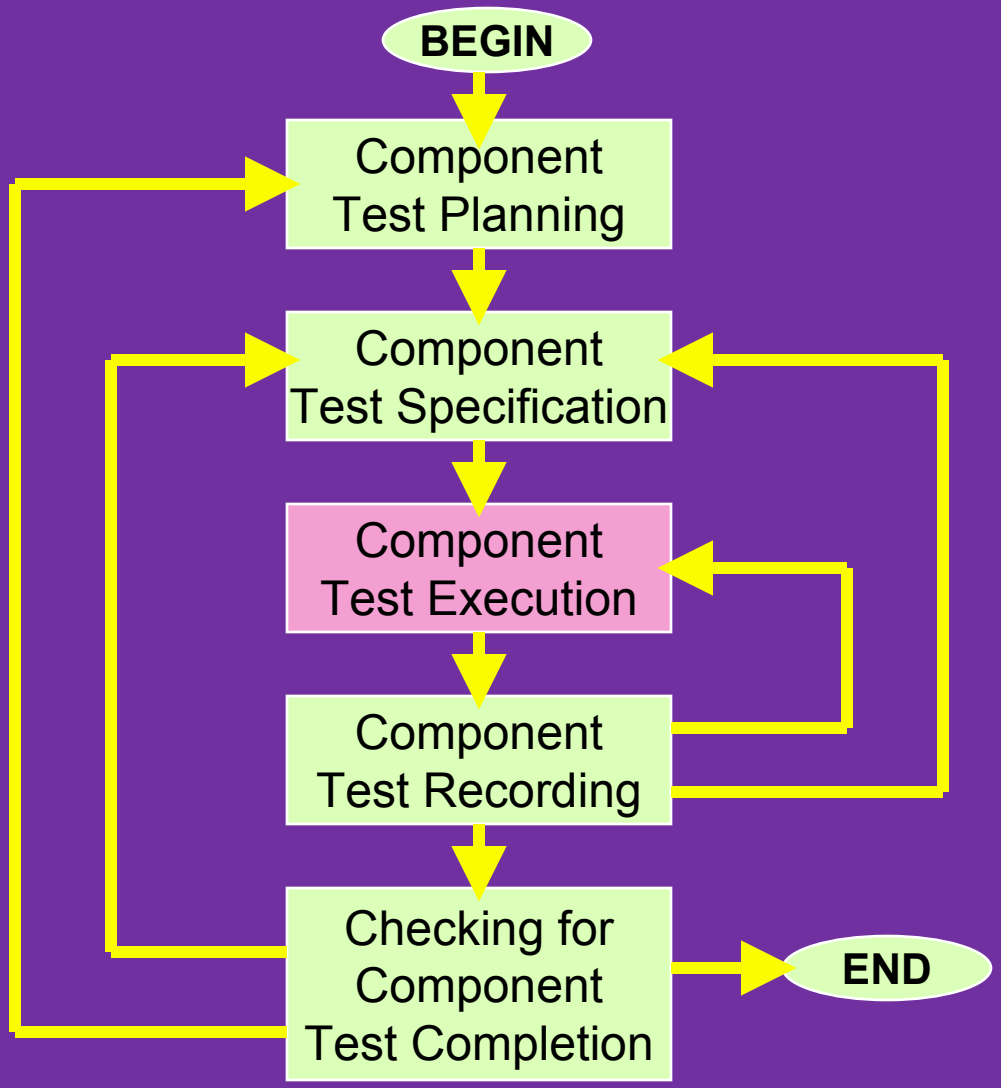
Component test process



Component test specification

- test cases are designed using the test case design techniques specified in the test plan (Section 3)
- Test case:
  - objective
  - initial state of component
  - input
  - expected outcome
- test cases should be repeatable

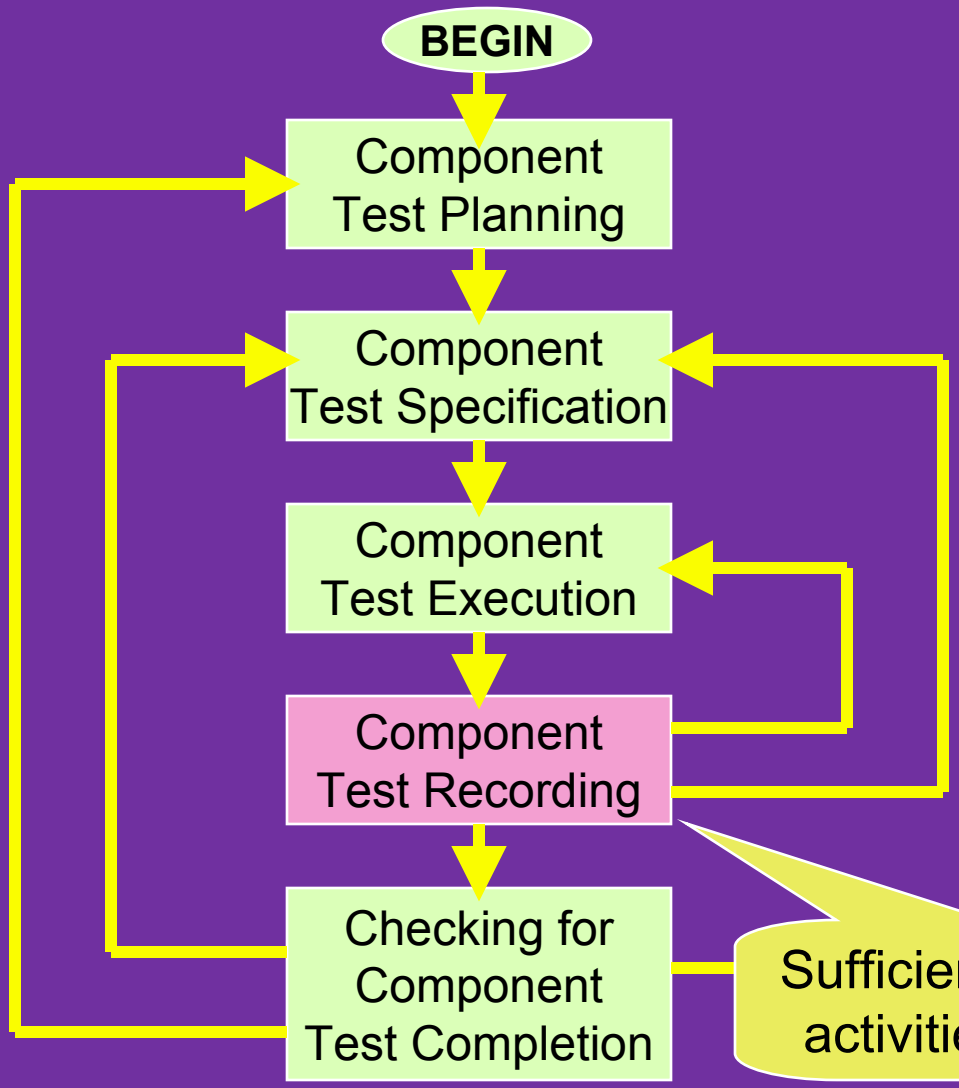
Component test process



Component test execution

- each test case is executed
- standard does not specify whether executed manually or using a test execution tool

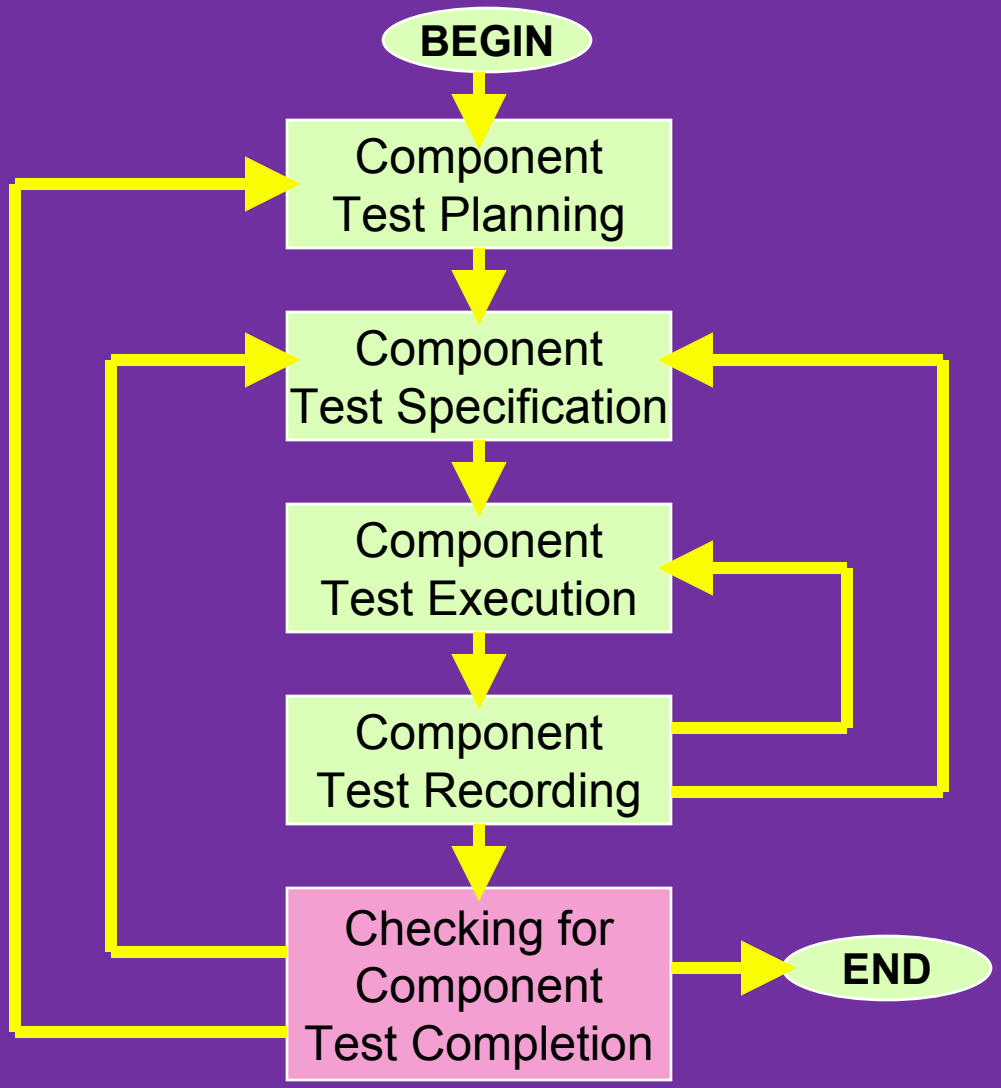
Component test process



Component test recording

- identities & versions of component, test specification
- actual outcome recorded & compared to expected outcome
- discrepancies logged
- repeat test activities to establish removal of the discrepancy (fault in test or verify fix)
- record coverage levels achieved for test completion criteria specified in test plan

Component test process



Checking for component test completion

- check test records against specified test completion criteria
- if not met, repeat test activities
- may need to repeat test specification to design test cases to meet completion criteria (e.g. white box)

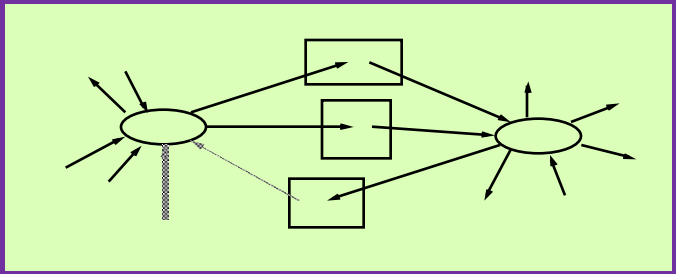
Test design techniques

Also a measurement technique? ✓ = Yes ✗ = No

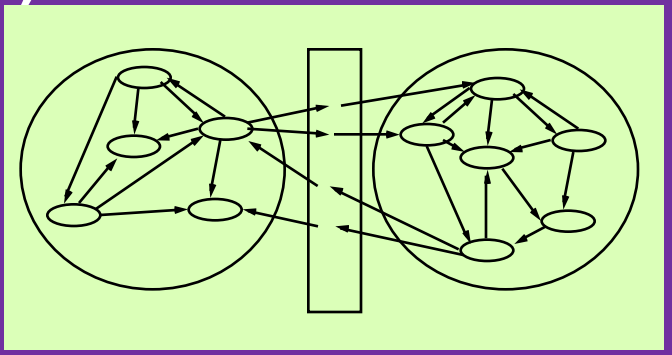
- **"Black box"**
  - Equivalence partitioning ✓
  - Boundary value analysis ✓
  - State transition testing ✓
  - Cause-effect graphing ✓
  - Syntax testing ✗
  - Random testing ✗
- **How to specify other techniques**

- **"White box"**
  - Statement testing ✓
  - Branch / Decision testing ✓
  - Data flow testing ✓
  - Branch condition testing ✓
  - Branch condition combination testing ✓
  - Modified condition decision testing ✓
  - LCSAJ testing ✓

Integration testing in the small



- more than one (tested) component
- communication between components
- what the set can perform that is not possible individually
- non-functional aspects if possible
- integration strategy: big-bang vs incremental (top-down, bottom-up, functional)
- done by designers, analysts, or independent testers



Big-Bang Integration

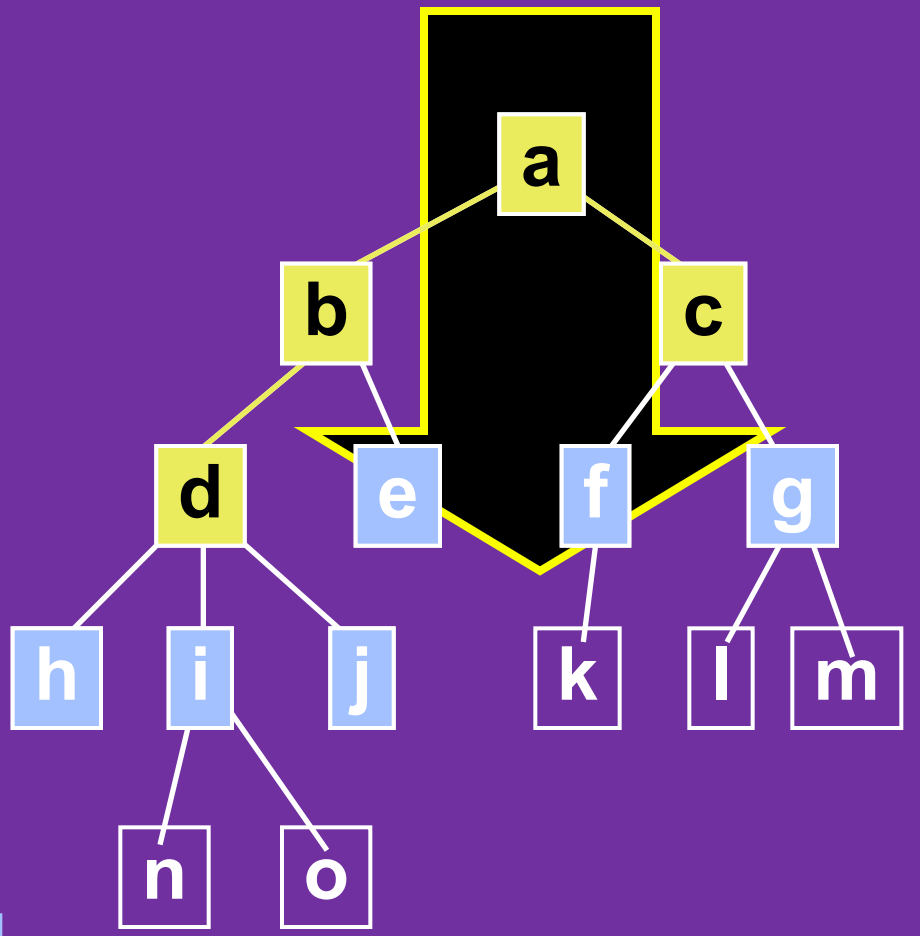
- **In theory:**
  - if we have already tested components why not just combine them all at once? Wouldn't this save time?
  - (based on false assumption of no faults)
- **In practice:**
  - takes longer to locate and fix faults
  - re-testing after fixes more extensive
  - end result? takes more time

Incremental Integration

- **Baseline 0: tested component**
- **Baseline 1: two components**
- **Baseline 2: three components, etc.**
- **Advantages:**
  - easier fault location and fix
  - easier recovery from disaster / problems
  - interfaces should have been tested in component tests, but ..
  - add to tested baseline

Top-Down Integration

- **Baselines:**
  - baseline 0: component a
  - baseline 1: a + b
  - baseline 2: a + b + c
  - baseline 3: a + b + c + d
  - etc.
- **Need to call to lower level components not yet integrated**
- **Stubs: simulate missing components**



Stubs

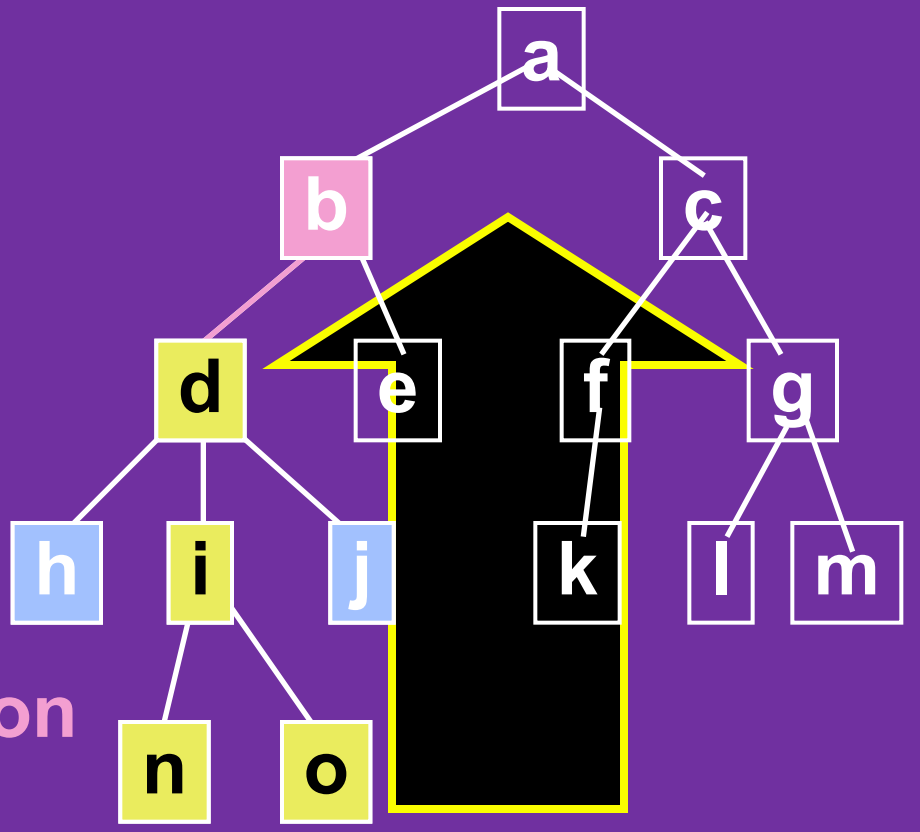
- **Stub (Baan: dummy sessions) replaces a called component for integration testing**
- **Keep it Simple**
  - print/display name (I have been called)
  - reply to calling module (single value)
  - computed reply (variety of values)
  - prompt for reply from tester
  - search list of replies
  - provide timing delay

Pros & cons of top-down approach

- **Advantages:**
  - critical control structure tested first and most often
  - can demonstrate system early (show working menus)
- **Disadvantages:**
  - needs stubs
  - detail left until last
  - may be difficult to "see" detailed output (but should have been tested in component test)
  - may look more finished than it is

Bottom-up Integration

- **Baselines:**
  - baseline 0: component n
  - baseline 1: n + i
  - baseline 2: n + i + o
  - baseline 3: n + i + o + d
  - etc.
- **Needs drivers to call the baseline configuration**
- **Also needs stubs for some baselines**



Drivers

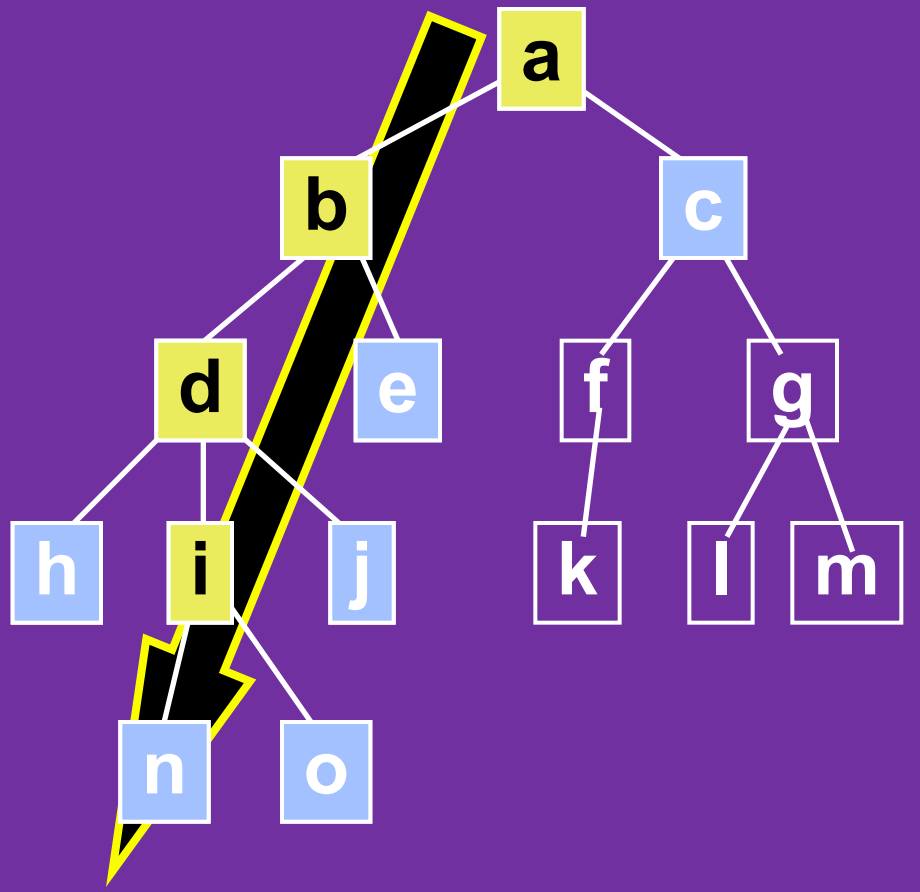
- **Driver (Baan: dummy sessions): test harness: scaffolding**
- **specially written or general purpose (commercial tools)**
  - invoke baseline
  - send any data baseline expects
  - receive any data baseline produces (print)
- **each baseline has different requirements from the test driving software**

Pros & cons of bottom-up approach

- **Advantages:**
  - lowest levels tested first and most thoroughly (but should have been tested in unit testing)
  - good for testing interfaces to external environment (hardware, network)
  - visibility of detail
- **Disadvantages**
  - no working system until last baseline
  - needs both drivers and stubs
  - major control problems found last

Minimum Capability Integration (also called Functional)

- **Baselines:**
  - baseline 0: component a
  - baseline 1: a + b
  - baseline 2: a + b + d
  - baseline 3: a + b + d + i
  - etc.
- **Needs stubs**
- **Shouldn't need drivers (if top-down)**

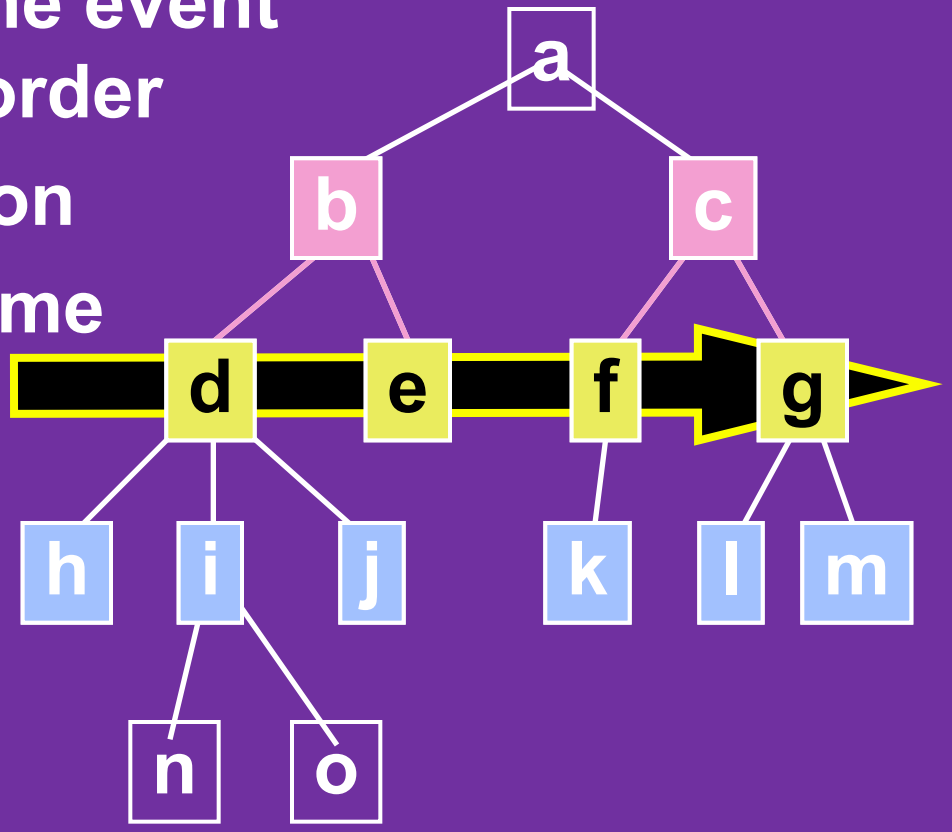


Pros & cons of Minimum Capability

- **Advantages:**
  - control level tested first and most often
  - visibility of detail
  - real working partial system earliest
- **Disadvantages**
  - needs stubs

Thread Integration (also called functional)

- order of processing some event determines integration order
- interrupt, user transaction
- minimum capability in time
- advantages:
  - critical processing first
  - early warning of performance problems
- disadvantages:
  - may need complex drivers and stubs



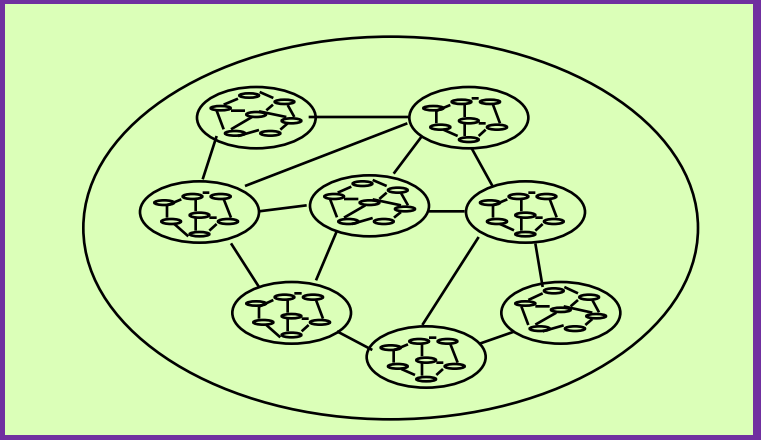
Integration Guidelines

- **minimise support software needed**
- **integrate each component only once**
- **each baseline should produce an easily verifiable result**
- **integrate small numbers of components at once**
  - one at a time for critical or fault-prone components
  - combine simple related components

Integration Planning

- **integration should be planned in the architectural design phase**
- **the integration order then determines the build order**
  - components completed in time for their baseline
  - component development and integration testing can be done in parallel - saves time

System testing



- **last integration step**
- **functional**
  - functional requirements and requirements-based testing
  - business process-based testing
- **non-functional**
  - as important as functional requirements
  - often poorly specified
  - must be tested
- **often done by independent test group**

Functional system testing

- **Functional requirements**
  - a requirement that specifies a function that a system or system component must perform (ANSI/IEEE Std 729-1983, Software Engineering Terminology)
- **Functional specification**
  - the document that describes in detail the characteristics of the product with regard to its intended capability (BS 4778 Part 2, BS 7925-1)

Requirements-based testing

- **Uses specification of requirements as the basis for identifying tests**
  - table of contents of the requirements spec provides an initial test inventory of test conditions
  - for each section / paragraph / topic / functional area,
    - risk analysis to identify most important / critical
    - decide how deeply to test each functional area

Business process-based testing

- **Expected user profiles**
  - what will be used most often?
  - what is critical to the business?
- **Business scenarios**
  - typical business transactions (birth to death)
- **Use cases**
  - prepared cases based on real situations

Non-functional system testing

- **different types of non-functional system tests:**
  - usability
  - security
  - documentation
  - storage
  - volume
  - configuration / installation
  - reliability / qualities
  - back-up / recovery
  - performance, load, stress

Performance Tests

- **Timing Tests**
  - response and service times
  - database back-up times
- **Capacity & Volume Tests**
  - maximum amount or processing rate
  - number of records on the system
  - graceful degradation
- **Endurance Tests (24-hr operation?)**
  - robustness of the system
  - memory allocation

Multi-User Tests

- **Concurrency Tests**
  - small numbers, large benefits
  - detect record locking problems
- **Load Tests**
  - the measurement of system behaviour under realistic multi-user load
- **Stress Tests**
  - go beyond limits for the system - know what will happen
  - particular relevance for e-commerce

Source: Sue Atkins, Magic Performance Management

Usability Tests

- messages tailored and meaningful to (real) users?
- coherent and consistent interface?
- sufficient redundancy of critical information?
- within the "human envelope"? (7±2 choices)
- feedback (wait messages)?
- clear mappings (how to escape)?

Who should design / perform these tests?

Security Tests

- passwords
- encryption
- hardware permission devices
- levels of access to information
- authorisation
- covert channels
- physical security

Configuration and Installation

- **Configuration Tests**
  - different hardware or software environment
  - configuration of the system itself
  - upgrade paths - may conflict
- **Installation Tests**
  - distribution (CD, network, etc.) and timings
  - physical aspects: electromagnetic fields, heat, humidity, motion, chemicals, power supplies
  - uninstall (removing installation)

Reliability / Qualities

- **Reliability**
  - "system will be reliable" - how to test this?
  - "2 failures per year over ten years"
  - Mean Time Between Failures (MTBF)
  - reliability growth models
- **Other Qualities**
  - maintainability, portability, adaptability, etc.

Back-up and Recovery

- **Back-ups**
  - computer functions
  - manual procedures (where are tapes stored)
- **Recovery**
  - real test of back-up
  - manual procedures unfamiliar
  - should be regularly rehearsed
  - documentation should be detailed, clear and thorough

Documentation Testing

- **Documentation review**
  - check for accuracy against other documents
  - gain consensus about content
  - documentation exists, in right format
- **Documentation tests**
  - is it usable? does it work?
  - user manual
  - maintenance documentation

Integration testing in the large

- **Tests the completed system working in conjunction with other systems, e.g.**
  - LAN / WAN, communications middleware
  - other internal systems (billing, stock, personnel, overnight batch, branch offices, other countries)
  - external systems (stock exchange, news, suppliers)
  - intranet, internet / www
  - 3rd party packages
  - electronic data interchange (EDI)

Approach

- **Identify risks**
  - which areas missing or malfunctioning would be most critical - test them first
- **“Divide and conquer”**
  - test the outside first (at the interface to your system, e.g. test a package on its own)
  - test the connections one at a time first (your system and one other)
  - combine incrementally - safer than “big bang” (non-incremental)

Planning considerations

- **resources**
  - identify the resources that will be needed (e.g. networks)
- **co-operation**
  - plan co-operation with other organisations (e.g. suppliers, technical support team)
- **development plan**
  - integration (in the large) test plan could influence development plan (e.g. conversion software needed early on to exchange data formats)

User acceptance testing

- **Final stage of validation**
  - customer (user) should perform or be closely involved
  - customer can perform any test they wish, usually based on their business processes
  - final user sign-off
- **Approach**
  - mixture of scripted and unscripted testing
  - ‘Model Office’ concept sometimes used

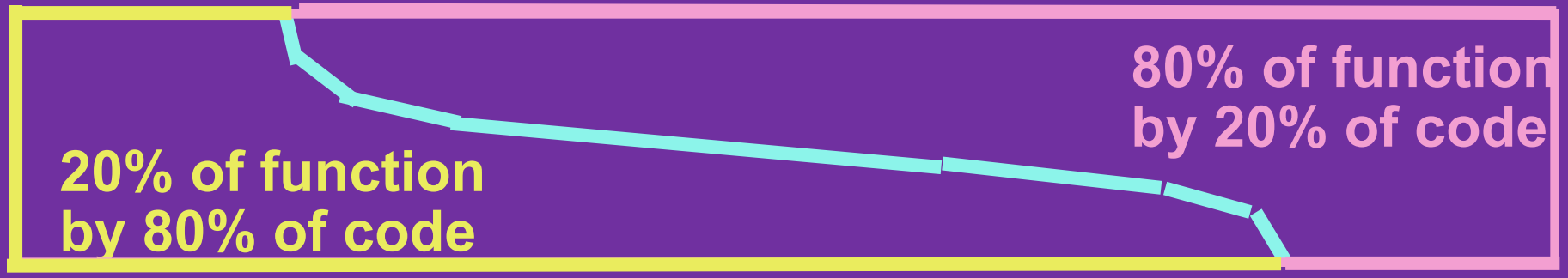
Why customer / user involvement

- **Users know:**
  - what really happens in business situations
  - complexity of business relationships
  - how users would do their work using the system
  - variants to standard tasks (e.g. country-specific)
  - examples of real cases
  - how to identify sensible work-arounds

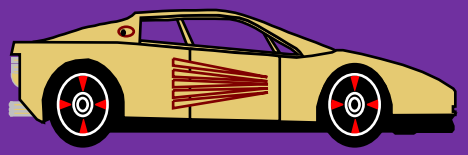
Benefit: detailed understanding of the new system

User Acceptance testing

Acceptance testing distributed over this line



System testing distributed over this line



Contract acceptance testing

- **Contract to supply a software system**
  - agreed at contract definition stage
  - acceptance criteria defined and agreed
  - may not have kept up to date with changes
- **Contract acceptance testing is against the contract and any documented agreed changes**
  - not what the users wish they had asked for!
  - this system, not wish system

Alpha and Beta tests: similarities

- **Testing by [potential] customers or representatives of your market**
  - not suitable for bespoke software
- **When software is stable**
- **Use the product in a realistic way in its operational environment**
- **Give comments back on the product**
  - faults found
  - how the product meets their expectations
  - improvement / enhancement suggestions?

Alpha and Beta tests: differences

- **Alpha testing**
  - simulated or actual operational testing at an in-house site not otherwise involved with the software developers (i.e. developers' site)
- **Beta testing**
- operational testing at a site not otherwise involved with the software developers (i.e. testers' site, their own location)

Acceptance testing motto

If you don't have patience to test the system

the system will surely test your patience

Lifecycle

1	2	3
4	5	6

Contents

Models for testing
Test level
Test types
Maintenance testing

Test types

- Testing of function (functional testing)
- Testing of software product characteristics (non-functional testing)
  - Functionality, reliability, usability, efficiency, maintainability, portability
- Testing of software structure/architecture (structural testing)
- Testing related to changes (confirmation and regression testing)

Lifecycle

1	2	3
4	5	6

Contents

Models for testing
Test level
Test types
Maintenance testing

Maintenance testing

- **Testing to preserve quality:**
  - different sequence
    - development testing executed bottom-up
    - maintenance testing executed top-down
    - different test data (live profile)
  - breadth tests to establish overall confidence
  - depth tests to investigate changes and critical areas
  - predominantly regression testing

What to test in maintenance testing

- **Test any new or changed code**
- **Impact analysis**
  - what could this change have an impact on?
  - how important is a fault in the impacted area?
  - test what has been affected, but how much?
    - most important affected areas?
    - areas most likely to be affected?
    - whole system?
- **The answer: “It depends”**

Poor or missing specifications

- **Consider what the system should do**
  - talk with users
- **Document your assumptions**
  - ensure other people have the opportunity to review them
- **Improve the current situation**
  - document what you do know and find out
- **Track cost of working with poor specifications**
  - to make business case for better specifications

What should the system do?

- **Alternatives**
  - the way the system works now must be right (except for the specific change) - use existing system as the baseline for regression tests
  - look in user manuals or guides (if they exist)
  - ask the experts - the current users
- **Without a specification, you cannot really test, only explore. You can validate, but not verify.**

Lifecycle

1	2	3
4	5	6

Summary: Key Points

V-model shows test levels, early test design

High level test planning

Component testing using the standard

Integration testing in the small: strategies

System testing (non-functional and functional)

Integration testing in the large

Acceptance testing: user responsibility

Maintenance testing to preserve quality